1 Set theory

This chapter overlaps with sections 2.1, 2.2, 2.3, 9.1 and 9.3 of Rosen.

1.1 Sets

1.1.1 Definitions

Definition 1.1 (Set and elements)

A set is a collection of objects which are called elements of the set. A set is fully determined by *which* elements compose it.

Notation

- If a is an element of the set A, we write $a \in A$.
- If an object a is not an element of the set A, we write $a \notin A$.

How to describe a set?

- By using a roster, listing all elements of the set, either explicitly like:

  $$S_1 = \{1, 2, 3, 4, 5, 6\}$$

  or implicitly, using an ellipsis:

  $$S_1 = \{1, 2, ..., 6\}$$

- By using a defining predicate:

  $$S = \{y \in S_0 \mid P(y)\}$$

  where $P(y)$ is a condition on the elements y of another set S_0. Then S is the set of all elements y of S_0 such that $P(y)$ is true. For instance:

  $$S_2 = \{y \in S_1 \mid y < 3\}$$

- Through a transformation f of the elements y of another set S_0:

  $$S = \{f(y) \mid y \in S_0\}$$

  For instance:

  $$S_3 = \{n^3 \mid n \in S_1\}$$

The notation "|" is read as "such that". It can also be written with a colon ":".

You may also combine the predicate and transformation methods, but it's very rarely necessary.

Remark

Note that there is often more than one way to define the same set. However, there is often a better choice, at least in terms of clarity.

Example. For instance, how would you write S_3 above using a predicate on the set of natural numbers?

Solution.

$$S_3 = \{m \in \mathbb{N} \mid \exists k \in \mathbb{N} \text{ such that } m = k^3\}$$

which is equally valid but probably less clear.

> **Question**
>
> From Definition 1.1, especially its second part, can you guess the condition for two sets to be equal? For instance, are $A = \{1, 2\}$, $B = \{2, 1\}$ and $C = \{1, 1, 2\}$ equal?

Solution. Yes they are! This means that the order of the elements in a set is irrelevant, as well as the number of occurrences of an element in the list.

> **Definition 1.2 (Sets equality)**
>
> Two sets are equal if and only if they have the same elements.

This definition of sets equality actually also indirectly defines what a set is.

> **Definition 1.3 (Empty set $\emptyset$)**
>
> The **empty set** $\emptyset$ is the set with no elements: $\emptyset = \{\}$.

> **Definition 1.4 (Universal set U)**
>
> The **universal set** U is the set containing all objects under consideration.

This concept can be useful to keep notation consistent. For instance, to formally write sets equality: two sets A and B are equal if and only if

$$\forall x \in U, x \in A \Leftrightarrow x \in B,$$

instead of

$$\forall x \in ?, x \in A \Leftrightarrow x \in B.$$

Most of the time, and in the following of this course, $x \in U$ can be omitted for convenience, though.

> **Definition 1.5 (Set finiteness and cardinality)**
>
> If there are exactly $n \in \mathbb{N}$ distinct elements in a set S, we say that S is a **finite set**, and that $|S| = n$ is the **cardinality** of S.

1.1.2 Examples: some already-known sets

> **Definition 1.6 (Natural numbers)**
>
> The set of natural numbers $\mathbb{N}$ is defined by the following conditions:
> (i) $0 \in \mathbb{N}$.
> (ii) If $n \in \mathbb{N}$, then the successor of n (i.e., the number $n + 1$) belongs to $\mathbb{N}$.

(iii) Every $n \in \mathbb{N}$ except 0 is the successor of some number in $\mathbb{N}$.
(iv) Every non-empty subset of $\mathbb{N}$ has a minimum element.

Remark

- Some define the natural numbers starting from one, thus excluding zero. To avoid any ambiguity, one can respectively use the terms "non-negative" or "positive" integers to distinguish between the definition of natural numbers including or excluding zero. Here, we will denote the set $\{n \in \mathbb{N} \mid n \neq 0\}$ as $\mathbb{N}^*$.
- We can informally "define" the following sets of numbers:
  - Integer numbers: $\mathbb{Z} = \{0, \pm 1, \pm 2, ...\}$
  - Rational numbers: $\mathbb{Q} = \left\{\frac{p}{q} \mid p, q \in \mathbb{Z}, q \neq 0\right\}$
- These sets are discrete yet infinite: their cardinality is undefined (or infinite).

1.1.3 Subsets

Definition 1.7 (Subset)

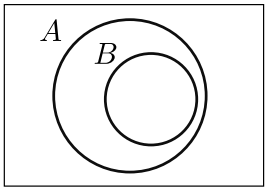
The set B is a **subset** of the set A ($B \subseteq A$) if and only if every element of B is also an element of A:

$$B \subseteq A \Leftrightarrow (b \in B \Rightarrow b \in A).$$

The set B is a **proper subset** of A ($B \subset A$) if B is a subset of A, and A contains at least an element not in B:

$$B \subset A \Leftrightarrow (b \in B \Rightarrow b \in A) \wedge (\exists a \in A, a \notin B).$$

We can represent $B \subseteq A$, with a **Venn diagram**:



Property 1.8

- The empty set $\emptyset$ is a subset of every set A: $\emptyset \subseteq A$.
- Every set A satisfies $A \subseteq A \subseteq U$.

Important

The latter implies that $A = B \Leftrightarrow (A \subseteq B) \wedge (B \subseteq A)$. This is the basis for how we usually prove the equality of two sets:
(i) Consider $x \in A$, prove it is also in B.
(ii) Consider $x \in B$, prove it is also in A.

Definition 1.9 (Power set)

The **power set** of the set A, denoted as $\mathcal{P}(A)$, is the set of all subsets of A:

$$\mathcal{P}(A) = \{B \mid B \subseteq A\}$$

1.2 Set operations

Given two sets A and B we can define a number of operations.

Definition 1.10 (Sets union)

The union of two sets A and B is the set of all elements that are in A, in B, or in both:

$$A \cup B = \{x \mid (x \in A) \vee (x \in B)\}$$

Definition 1.11 (Sets intersection)

The intersection of two sets A and B is the set of all elements that are in both A and B:

$$A \cap B = \{x \mid (x \in A) \wedge (x \in B)\}$$

Notation

These two operations can be chained an arbitrary number of times.
For example we can perform the union of n sets $A_1, A_2, ..., A_n$:

$$A_1 \cup A_2 \cup ... \cup A_n = \bigcup_{i=1}^{n} A_i,$$

or perform the intersection of the same sets:

$$A_1 \cap A_2 \cap ... \cap A_n = \bigcap_{i=1}^{n} A_i.$$

These two notations are the respective equivalents of the sum $\sum$ and product $\prod$ notations that you already knew.

Definition 1.12 (Sets difference)

The difference of two sets A and B is the set of all elements that are in A but not in B:

$$A \setminus B = \{x \mid (x \in A) \wedge (x \notin B)\}$$

Definition 1.13 (Set complement)

The complement of a set A is the set of all elements that are not in A:

$$\overline{A} = \{x \mid x \notin A\} = U \setminus A$$

Definition 1.14 (Sets symmetric difference)

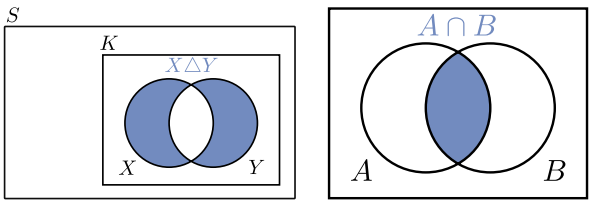
The symmetric difference of two sets A and B is the set of all elements that are in A or in B but not in both:

$$A \triangle B = \{x \mid (x \in A \cup B) \wedge (x \notin A \cap B)\}$$

A set can therefore be defined as the result of operations involving other sets. For instance,

$$\mathbb{Z} = \mathbb{N} \cup \{-n \mid n \in \mathbb{N}\}, \quad \mathbb{N}^* = \mathbb{N} \setminus \{0\}$$

You can represent these operations with a Venn diagram too. For instance:



Question

How would you represent the other operations? Then, using these representations, how would you write the difference operations as combinations of the union, intersection and complement operations?

Solution.

$$A \setminus B = A \cap \left(\overline{B}\right),$$
$$A \triangle B = (A \cup B) \setminus (A \cap B).$$

Property 1.15

- **Distributive laws**:
  - $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
  - $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
- **De Morgan's laws**:
  - $\overline{A \cup B} = \overline{A} \cap \overline{B}$
  - $\overline{A \cap B} = \overline{A} \cup \overline{B}$
- $A \triangle B = (A \setminus B) \cup (B \setminus A)$

You can check these laws hold with Venn diagrams too.

Definition 1.16 (Disjoint sets)

Two sets A and B are **disjoint** if $A \cap B = \emptyset$.

1.3 Applications

1.3.1 Data types as sets

The very concept of a datatype in computer science is built upon the definition of a set. For instance, a boolean is an element of the set $\{0, 1\}$, or, equivalently, `{False, True}`. A datatype is more than a set though, as it also defines the operations which are allowed on the elements of the set.

> **Question**
>
> Note that all data types which can be defined on computers correspond to a set which is *necessarily* finite. Can you guess why?

1.3.2 Sets as a data type

Most programming languages define a set datatype. Python is no exception: you can define a set with a syntax that is very similar to the notation we have seen until now:

```
S = {'a', 1, {'b'}}
```

> **Check at home**
>
> Find the equivalents of the set operations we saw above in the Python documentation, and try them out! Also check that sets in Python have the properties listed above, in particular that the ordering of elements does not matter.

1.4 Relations

> **Definition 1.17 (Tuple)**
>
> An n-tuple is an ordered collection of n objects, of the form $(a_1, a_2, ..., a_n)$. A 2-tuple is often called an ordered pair.

This means that, contrary to sets, for tuples the ordering of elements matters. Thus, while $\{a_1, a_2\} = \{a_2, a_1\}$, for tuples $(a_1, a_2) \neq (a_2, a_1)$ if $a_1 \neq a_2$.

> **Definition 1.18 (Cartesian product)**
>
> Given two sets A and B, the **Cartesian product** $A \times B$ is the set of all ordered pairs (a, b) where $a \in A$ and $b \in B$. Formally, that is:
>
> $$A \times B = \{(a, b) \mid (a \in A) \land (b \in B)\}$$

> **Remark**
>
> A Cartesian product gives you all the possible combinations involving one element from each set. If $A = B$, it gives all the possible combinations of two elements in this set.
>
> You already saw Cartesian products in calculus, when you wrote, for instance: $\forall (x, y) \in \mathbb{R}^2$: these are all the possible pairs of real numbers. Then, you used the notation $\mathbb{R}^2 = \mathbb{R} \times \mathbb{R}$.

The Cartesian product allows us to introduce a fundamental concept of discrete mathematics: relations.

Definition 1.19 (Binary relations)

A **binary relation** R from the set A to the set B is a set of ordered pairs from the two sets. Said differently, R is a subset of $A \times B$, so:

$$R \subseteq A \times B.$$

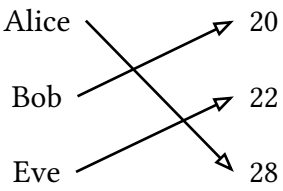
A is then called the **domain** of R, while B is called its **codomain**.

If $a \in A$ is related to $b \in B$ by R, we therefore have $(a, b) \in R$, which we can also write $a\,R\,b$. If they are not related, we have $(a, b) \notin R$, which we can also write $a \not R\, b$.

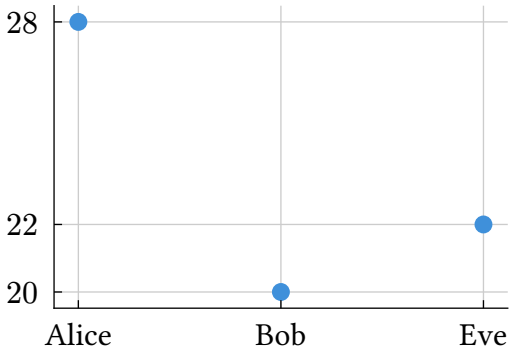
How can we represent binary relations graphically?

What we want is to show each set on a separate "side", and link related elements together. Here we'll see three options, which are the directed graph, Cartesian and adjacency matrix representations.

Example. Let's consider that we know three persons, called Alice, Bob and Eve, who are respectively 28, 20 and 22 years-old. We want to represent this information as a binary relation. We can then consider the set $A = \{\text{Alice}, \text{Bob}, \text{Eve}\}$ of people's names and the set $B = \{20, 22, 28\}$ of some ages. We can thus define the relation R as $a\,R\,b$ if person $a \in A$ has the age $b \in B$, and represent it as:



$$\begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$



Question

- When is the Cartesian representation preferable? Hint: imagine if we took $B = \mathbb{N}$.
- Imagine other relations in which the individual relations are not one-to-one, but one-to-many, many-to-one or many-to-many.

To be more formal:

Definition 1.20 (Adjacency matrix)

Let's consider a relation R from the set A to the set B, and the orderings $a_1, a_2, \ldots, a_{|A|}$ and $b_1, b_2, \ldots, b_{|B|}$ of these two sets. The **adjacency matrix** of R associated to these orderings is the $|A| \times |B|$ matrix M whose entries satisfy

$$M_{ij} = \begin{cases} 1 \text{ if } a_i \, R \, b_j \\ 0 \text{ if } a_i \not R \, b_j \end{cases}$$

We clearly saw graphically that relations have a direction, that one can naturally reverse.

Definition 1.21 (Inverse relations)

Let R be a relation from the set A to the set B. The inverse relation of R is the relation that assigns to each element $b \in B$ an element $a \in A$. We denote the inverse by R^{-1}, so that

$$\forall (a, b) \in A \times B, a \, R \, b \Leftrightarrow b \, R^{-1} \, a$$

Remark

All relations have an inverse.

Definition 1.22 (Composition of relations)

Let R be a relation from the set A to the set B, and let S be a relation from the set B to the set C. The composition of the relation S and R is a relation from A to C denoted as $S \circ R$. In particular, $S \circ R$ is a subset of the Cartesian product $A \times C$ such that, given any $a \in A$ and $c \in C$, $a(S \circ R)c$ if and only if there exists some $b \in B$ satisfying $a \, R \, b$ and $b \, S \, c$.

Example. We can consider a set A corresponding to the members of your family from your generation, a set B corresponding to the previous generation, and C to two generation ago. We can introduce the relations

$$R = \{(a, b) \in A \times B \mid a \text{ is the child of } b\},$$
$$S = \{(b, c) \in B \times C \mid b \text{ is the child of } c\},$$

whose composition gives

$$\begin{aligned} S \circ R &= \{(a, c) \in A \times C \mid \text{a parent of } a \text{ is the child of } c\} \\ &= \{(a, c) \in A \times C \mid a \text{ is the grandchild of } c\}. \end{aligned}$$

Proposition 1.23

Let M_R be the adjacency matrix of the relation R from the set A to the set B, and let M_S be the adjacency matrix of the relation S from the set B to the set C. Then, the adjacency matrix $M_{S \circ R}$ of the composition of the relations $S \circ R$ from A to C is given by:

$$M_{S \circ R} = M_R \odot M_S$$

where the product $\odot$ is the Boolean product of matrices.

Using Boolean operations (instead of regular ones) guarantees that $M_{S \circ R}$ is an adjacency matrix associated to a binary relation.

1.5 Defining functions as relations

Definition 1.24 (Functions)

A binary relation f with a domain X and codomain Y is said to be a **function** if it assigns exactly one element of Y to each element of X.

Notation

A function $f \subseteq X \times Y$ is usually defined with the notation $f : X \to Y$, to signify that it maps each element of X to a unique element of Y.

Also, we write $f(x) = y$ if y is the unique element of Y assigned by f to the element x of X.

Definition 1.25 (Function images and range)

Let's consider $f : X \to Y$ and $(x, y) \in X \times Y$ such that $f(x) = y$. We then say that y is the **image** of x by f, and that x is the **preimage** of y by f. We also define the **range** of the function as the set of images of every element in the domain X, and often write this range as $f(X)$. The range is therefore a subset of the codomain: $f(X) \subseteq Y$.

Definition 1.26

Given a function $f : X \to Y$, we say that

- f is **injective** or **one-to-one** if every element of Y is the image of at most one element of X by f, that is:

$$\forall x_1, x_2 \in X, f(x_1) = f(x_2) \Rightarrow x_1 = x_2$$

- f is **surjective** if every element of Y is the image of at least one element of X by f, that is:

$$\forall y \in Y, \exists x \in X, y = f(x).$$

- f is **bijective** if it is injective and surjective.

Definition 1.27 (Inverse function)

If $f : X \to Y$ is bijective, we can define its **inverse function** $f^{-1} : Y \to X$ by the well-defined rule:

$$f^{-1}(y) = x \iff y = f(x)$$

Question

While all relations have an inverse, a function, which is a particular kind of relation, only has an inverse if it is bijective. How come?

Definition 1.28 (Composition of functions)

Given two functions $f : X \to Y$, $g : Y \to Z$, we can define a new function $g \circ f : X \to Z$ by the following rule:

$$(g \circ f)(x) = g(f(x))$$

The function $g \circ f$ is the **composition** of f and g.

2 Relations

This chapter overlaps with sections 9.1, 9.5, 9.6, 5.1 and 5.2 of Rosen.

2.1 Binary relations on a set

While we previously introduced relations R from a set A to a set B, here we'll be particularly interested in the case where $A = B$, that is relations on a single set.

Definition 2.1 (Binary relation on a set)

A **binary relation** R on the set A is a subset of $A \times A$, so:

$$R \subseteq A \times A.$$

In the following, unless otherwise specified, "a relation R" refers to a relation on a single set A.

Let's now see some properties which will allow us to define two particular kinds of relations which are useful to compare elements of a set.

Definition 2.2 (Reflexive relations)

A relation R is reflexive if all elements of A are related to themselves:

$$\forall a \in A, a \, R \, a$$

A relation R is irreflexive if all elements of A are not related to themselves:

$$\forall a \in A, a \not R \, a$$

Definition 2.3 (Symmetric relations)

A relation R is symmetric if $R = R^{-1}$, i.e., if

$$\forall (a, b) \in A^2, a \, R \, b \Rightarrow b \, R \, a$$

A relation R is antisymmetric if

$$\forall (a, b) \in A^2 \text{ such that } a \neq b, a \, R \, b \Rightarrow b \not R \, a$$

Definition 2.4 (Transitive relations)

A relation R is transitive if

$$\forall (a, b, c) \in A^3, (a \, R \, b) \wedge (b \, R \, c) \Rightarrow a \, R \, c$$

This looks a lot like what we saw in compositions: in fact if $(a, b) \in R$ and $(b, c) \in R$, then by Definition 1.22, $(a, c) \in R \circ R$, hence:

Lemma 2.5

A relation R is transitive if and only if $(R \circ R) \subseteq R$.

This lemma can then be used to prove the following by induction.

Theorem 2.6

A relation R is transitive if and only if $R^n \subseteq R$ for all $n \in \mathbb{N}$. The n-th power R^n of the relation R is recursively defined as follows:

$$R^1 = R, \quad R^n = R \circ R^{n-1}$$

2.2 Equivalence relations

Definition 2.7 (Equivalence relations)

A relation R on a set A is an equivalence relation if it is reflexive, symmetric and transitive.

Notation

If R is an equivalence relation, $a \, R \, b$ is usually denoted as $a \equiv_R b$ or $a \sim_R b$.
The R subscript can be dropped when there is no ambiguity on which equivalence relation we're dealing with.

Question

What is the simplest equivalence relation that you know?

Example. Is the following relation on the set P of all people an equivalence relation?

$$R_P = \{(a, b) \in P^2 \mid a \text{ is the brother of } b\}$$

If not, what's the closest relation you can think of that's actually an equivalence?

Definition 2.8 (Equivalence classes)

Let R be an equivalence relation on a set A. The set of all the elements of A related to a certain element $a \in A$ is called the **equivalence class** of a with respect to R, and it is denoted as $[a]_R$, or simply as $[a]$. Therefore,

$$[a]_R = \{b \in A \mid a \, R \, b\}$$

Any element $b \in [a]_R$ is called a **representative** of the equivalence class of a.

Theorem 2.9

Let R be an equivalence relation on A. Then,
(i) $[a]_R$ is non-empty for all $a \in A$.
(ii) For any two elements $a, b \in A$, either $[a]_R = [b]_R$ (and $a \, R \, b$), or $[a]_R \cap [b]_R = \emptyset$.
(iii) The equivalence classes determine the relation uniquely.

Equivalence classes are useful because they allow us to partition a set. But first, what does that even mean?

Definition 2.10 (Set partition)

A partition of a set A is a set of disjoint nonempty subsets of A whose union form A.
In other words, a set of subsets $\{A_i\}_{i \in I}$ such that:

(i) $\forall i \in I, A_i \neq \emptyset$,

(ii) $\forall (i,j) \in I^2, A_i \cap A_j = \emptyset$ if $i \neq j$,

(iii) $\bigcup_{i \in I} A_i = A$,

and where I is a (potentially infinite!) set of indices.

Theorem 2.11

Let R be an equivalence relation on A.
Then the set of all equivalence classes of R form a partition of A.
Conversely, given a partition $\{A_i\}$ of A, there exists an equivalence relation R such that its equivalence classes are the sets A_i.

Question

If there exists $a, b \in A$ such that $a \equiv b$, then $[a]_R = [b]_R$, which means $[a]_R \cap [b]_R \neq \emptyset$. Since equivalence classes partition a set, doesn't that contradict the fact that the subsets composing a partition should be disjoint?

Example. From the equivalence relation you defined in the example above, translate in plain words what its associated equivalence classes represent. Then also explain in plain words why they do form a partition of the set of all people, that is how they fulfill each condition of Definition 2.10.

Definition 2.12 (Quotient set)

Let R be an equivalence relation on A. The set of all the equivalence classes of R is called the **quotient set** of A by R, and it is denoted by A/R:

$$A/R = \{[a]_R \mid a \in A\}$$

Following Theorem 2.11, the quotient set A/R can also be called the **partition** of A by R.

2.3 Order relations

2.3.1 Partial and total orders

Definition 2.13 (Partial order relation)

A relation on a set A is called a **partial order** if it is reflexive, antisymmetric, and transitive.

Notation

Order relations are usually denoted by the symbol $\preccurlyeq$.
We write $a \prec b$ to denote that $a \preccurlyeq b$ and $a \neq b$, and then say that "a precedes b".

Definition 2.14 (Posets)

A set A equipped with an order relation $\preccurlyeq$ is called a **partially ordered set**, or poset, and is denoted by $(A, \preccurlyeq)$.

Example. Are the following posets?
(i) $(\mathbb{Z}, \geq)$
(ii) $(\mathbb{N}^*, |)$, where $|$ is the "divides" relation:

$$a \mid b \Leftrightarrow \exists q \in \mathbb{N} \text{ such that } b = q \cdot a.$$

Since a partial order is a relation, by definition it does not impose any constraint between unrelated elements. It is therefore useful to distinguish between elements which can be compared using an ordering, and those which cannot.

Definition 2.15 (Comparability)

Let $(A, \preccurlyeq)$ be a partially ordered set. Two elements $a, b \in A$ are said to be **comparable** if either $a \preccurlyeq b$ or $b \preccurlyeq a$. If neither of these conditions holds, such elements are said to be incomparable.

The adjective "partial" in "partial ordering" thus refers to the fact that the ordering does not necessarily order all elements of its associated set. But if it is the case, the poset has some interesting properties, and so we have a special term to refer to them.

Definition 2.16 (Total order)

A partially ordered set $(A, \preccurlyeq)$ is said to be **totally ordered** when any two elements of A are comparable.

2.3.2 Representing posets: Hasse diagrams

We saw in Section 1.4 that relations can be represented by a directed graph, by drawing an arrow from $a \in A$ to $b \in B$ if $a \, R \, b$. A poset can therefore also be represented as a directed graph, but with several simplifications due to its properties.

(i) It is a relation on a single set ($A = B$), so elements only need to be represented once.
(ii) It is antisymmetric, so we can fix a convention such as "if $a \prec b$, then a will be represented below b". This implies that all arrows point in the same direction, upwards, so they can be omitted.
(iii) It is reflexive, so the loops corresponding to $a \preccurlyeq a$ do not need to be represented.
(iv) It is transitive, so the edge corresponding to $a \prec c$ does not need to be represented if there exists b such that $a \prec b$ and $b \prec c$.

All these lead to a representation known as a **Hasse diagram**.

Example. Draw the Hasse diagram of $(\{1, 2, 3, 4\}, \leq)$.

2.3.3 Extremal elements

Definition 2.17 (Extremal elements)

Let $(A, \preccurlyeq)$ be a partially ordered set.

- $M \in A$ is a maximal element if it does not precede any other element:

$$\forall a \in A, M \preccurlyeq a \Rightarrow a = M$$

- $m \in A$ is a minimal element if no other element precedes it:

$$\forall a \in A, a \preccurlyeq m \Rightarrow a = m$$

In other words, in the Hasse diagram associated to $(A, \preccurlyeq)$, there is no element above M, and no element below m.

Definition 2.18 (The maximum and minimum)

Let $(A, \preccurlyeq)$ be a partially ordered set.

- $M^* \in A$ is **the maximum (or greatest element)** of A if all elements precede or are equal to it:

$$\forall a \in A, a \preccurlyeq M^*$$

- $m^* \in A$ is **the minimum (or least element)** if it precedes or is equal to all elements:

$$\forall a \in A, m^* \preccurlyeq a$$

In other words, in the Hasse diagram associated to $(A, \preccurlyeq)$, M^* is above all the others, and m^* is below them.

Notation

The maximum and minimum of $(A, \preccurlyeq)$ are denoted by $\max(A)$ and $\min(A)$, respectively.

Remark

The greatest (maximum) and least (minimum) elements might not exist for a given $(A, \preccurlyeq)$.

Example. What extremal elements do the following posets have? Do they have a maximum and minimum?

(i) $(\{2, 5, 3, 7\}, |)$,
(ii) $(\{2, 3, 4, 9\}, |)$,
(iii) $(\{2, 4, 6, 8\}, |)$,

("$|$" is the "divides" relation from the example above).

Theorem 2.19

The maximum M^* of a partially ordered set $(A, \preccurlyeq)$, if it exists, is unique. In addition, the maximum of $(A, \preccurlyeq)$ is also a maximal element of it.

Remark

To be the maximum, a maximal element needs to be comparable to all other elements!

Definition 2.20 (Bounds)

Let $(A, \preccurlyeq)$ be a partially ordered set, and $B \subset A$.

- $u \in A$ is an upper bound of B if $b \preccurlyeq u$ for all $b \in B$.
  - The set of the upper bounds of B is denoted by $\text{major}(B)$.
  - The supremum of B, $\sup(B)$, is the least upper element of B:

$$\sup(B) = \min(\text{major}(B))$$

- $l \in A$ is a lower bound of B if $l \preccurlyeq b$ for all $b \in B$.
  - The set of all the lower bounds of B is denoted by $\text{minor}(B)$.
  - The infimum of B, $\inf(B)$, is the greatest lower element of B:

$$\inf(B) = \max(\text{minor}(B))$$

Remark

It may happen that $\text{major}(B) = \emptyset$, $\text{minor}(B) = \emptyset$ and/or $\sup(B)$ and $\inf(B)$ do not exist.

2.3.4 Well-ordering and induction

Definition 2.21 (Well-ordered sets)

$(A, \preccurlyeq)$ is a well-ordered set if it is totally ordered and all nonempty subsets of A have a minimum.

Proposition 2.22

- The set of natural numbers with the usual order $(\mathbb{N}, \leq)$ is a well-ordered set.
- The totally-ordered set $(\mathbb{Z}, \leq)$ is not a well-ordered set; but as $\mathbb{Z}$ is isomorphic to $\mathbb{N}$, we can choose another order $\preccurlyeq$ such that $(\mathbb{Z}, \preccurlyeq)$ is a well-ordered set.

The well-ordering of natural numbers then explains the validity of proofs by induction!

Theorem 2.23 (Induction principle: weak version)

Let P be some predicate on the positive integers. If the following conditions are fulfilled:
Basis step We can show that $P(n_0)$ is true for a fixed $n_0 \in \mathbb{N}$.
Inductive step If we assume that $P(k)$ is true for some unspecified $k \geq n_0$, then we can show that $P(k+1)$ is true.

Then $P(n)$ is true for every $n \geq n_0$.

Theorem 2.24 (Induction principle: strong version)

Let P be some predicate on the positive integers. If the following conditions are fulfilled:

Basis step We can show that $P(n_0)$ is true for a fixed $n_0 \in \mathbb{N}$.
Inductive step If we assume that for some unspecified $k \geq n_0$, $P(j)$ is true for any $n_0 \leq j \leq k$, then we can show that $P(k+1)$ is true.

Then $P(n)$ is true for every $n \geq n_0$.

> **Remark**
>
> n_0 is very often 0 or 1, but not always!

Example. Find n_0 such that $2^n \geq n + 5$ for all $n \geq n_0$.

2.4 Summary: types of relations

Relation	Reflexive	Symmetric	Anti-symmetric	Transitive	Additional Properties
Equivalence	✅	✅	❌	✅	
Order	✅	❌	✅	✅	
Total order	✅	❌	✅	✅	Every pair comparable
Well-ordered set	✅	❌	✅	✅	Every subset $\neq \emptyset$ has a minimum

You will see different kinds of relations again in a more or less close future.

- In **relational databases**, each table defines a relation between n sets of attributes, where the values for each set are stored in a column (see Rosen 9.2 for further reading).
- A particular case of equivalence relation related to **modular arithmetic** and its associated classes will be studied in Section 3.2.
- Order relations constitute the **formalism** on which all **sorting algorithms** are built, whether they concern numbers or strings of text.

3 Elementary number theory

This chapter overlaps with sections 4.1, 4.3, 4.4, 4.5 and 4.6 of Rosen.

3.1 Integer divisibility

The set of integers $\mathbb{Z}$ is closed with respect to the operations of sum, subtraction, and product. In other words, for every $a, b \in \mathbb{Z}$, $a \pm b \in \mathbb{Z}$ and $a \cdot b \in \mathbb{Z}$. They also satisfy:

- 0 is the identity with respect to the sum: $a + 0 = a$ for every $a \in \mathbb{Z}$.
- 1 is the identity with respect to the product: $a \cdot 1 = a$ for every $a \in \mathbb{Z}$.
- For every $a \in \mathbb{Z}$, there exists a unique inverse element $-a \in \mathbb{Z}$ such that $a + (-a) = 0$.

However, the result of dividing two integers is not necessarily an integer!

Definition 3.1 (Divisibility)

Given two integers $a \neq 0$ and b, we say that a **divides** b if there exists a **quotient** $q \in \mathbb{Z}$ such that $b = a \cdot q$.

Then a is called a **factor** or **divisor** of b, and b a **multiple** of a.

We denote $a \mid b$ when a divides b, and we write $a \nmid b$ when a does not divide b. So, in summary:

$$a \mid b \Leftrightarrow \exists q \in \mathbb{Z}, b = a \cdot q$$

Remark

- Every nonzero integer divides 0:

$$\forall a \in \mathbb{Z} \setminus \{0\}, 0 = a \cdot 0 \quad (q = 0).$$

- Every nonzero integer divides itself:

$$\forall a \in \mathbb{Z} \setminus \{0\}, a = a \cdot 1 \quad (q = 1).$$

- 1 divides any integer:

$$\forall a \in \mathbb{Z}, a = 1 \cdot a \quad (q = a).$$

Theorem 3.2

Let a, b, c be integers. Then:

(i) If $a \mid b$ and $a \mid c$, then $a \mid (b + c)$.
(ii) If $a \mid b$, then $a \mid (b \cdot c)$ for every $c \in \mathbb{Z}$.
(iii) If $a \mid b$ and $b \mid c$, then $a \mid c$.
(iv) If $c \neq 0$, then $a \mid b$ if and only if $(c \cdot a) \mid (c \cdot b)$.
(v) If $a \mid b$ and $b \neq 0$, then $|a| \leq |b|$.
(vi) If $a \mid b$ and $b \mid a$, then $a = \pm b$.

Theorem 3.3

If $a \mid b_i$ for $i = 1, \ldots, N$, then $a \mid \sum_{i=1}^{N} u_i \cdot b_i$ for all $u_i \in \mathbb{Z}$. In other words, an integer divides any linear combination with integer coefficients of its multiples.

What if an integer does not divide another? Then we need to involve a remainder.

Theorem 3.4 (The Division Algorithm)

Let a and $b \neq 0$ be two integers. Then there exists a unique pair of integers q and r such that

$$a = q \cdot b + r \quad \text{with } 0 \leq r < |b|$$

Definition 3.5

In the equality given in Theorem 3.4:

- the numbers a and b are called dividend and divisor, respectively,
- the number r is called the **remainder** and we write

$$r = a \bmod b,$$

- the number q is called the **quotient** and we write

$$q = a \text{ div } b = \begin{cases} \lfloor a/b \rfloor & \text{if } b > 0 \\ \lceil a/b \rceil & \text{if } b < 0 \end{cases}.$$

Remark

mod is called the modulo or remainder operation, and div the floor division or quotient operation. They both have associated symbols in most programming languages (respectively `%` and `//` in Python, for instance).

3.2 Modular arithmetic

3.2.1 Congruence

Modular arithmetic allows us to perform algebraic operations using, instead of a given set of numbers, their respective remainders with respect to some fixed positive number called the modulus. This might sound convoluted but is actually fairly common. For example, to answer the question "What day of the week will we be 10 days from now?", you use modular arithmetic.

We've already seen the operation notation $r = a \bmod b$, but to perform modular arithmetic we need to introduce a related notation, which involves a relation instead.

Definition 3.6 (Congruence)

Let $a, b \in \mathbb{Z}$, and $m \in \mathbb{N}^*$. Then a and b are said to be congruent modulo m if $m \mid (a - b)$. This relation is denoted as $a \equiv b (\bmod m)$. It is called a congruence with modulus m.

$a \equiv b (\bmod m)$ and $a \bmod m = b$ include different uses of "mod"": in the first it represents a relation, while it represents an operation in the second. However, this shared notation comes from the fact that the relation and the operation are closely related.

Theorem 3.7

Let $a, b \in \mathbb{Z}$ and $m \in \mathbb{N}^*$. a and b are congruent modulo m if and only if they have the same remainder when divided by m, that is:

$$a \equiv b (\mathrm{mod}\, m) \Leftrightarrow a \bmod m = b \bmod m.$$

Question

- Can you now translate "What day of the week will we be 10 days from now?" in modular arithmetic terms? Start with wondering: how can I represent days of the week numerically? Then, what is the modulus, here?
- Can you find other examples of real-life problems involving congruences?

From the definition of divisibility, we can directly get that:

Theorem 3.8

Let $a, b \in \mathbb{Z}$ and $m \in \mathbb{N}^*$. a and b are congruent modulo m if and only if there is an integer q such that $a = q \cdot m + b$, that is:

$$a \equiv b (\mathrm{mod}\, m) \Leftrightarrow \exists q \in \mathbb{Z}, a = q \cdot m + b.$$

...so a congruence is just another way of writing the division algorithm (Theorem 3.4), where the quotient is disregarded.

Let's now see how operations between congruent integers go.

Theorem 3.9

Let m be a positive integer. If $a_1 \equiv b_1 (\mathrm{mod}\, m)$ and $a_2 \equiv b_2 (\mathrm{mod}\, m)$, then:

- $a_1 \pm a_2 \equiv b_1 \pm b_2 (\mathrm{mod}\, m)$.
- $a_1 \cdot a_2 \equiv b_1 \cdot b_2 (\mathrm{mod}\, m)$.

Proof: Using Theorem 3.8, we know there exist s and t such that $a_1 = b_1 + sm$ and $a_2 = b_2 + tm$, so:

$$\begin{cases} a_1 + a_2 = (b_1 + b_2) + (s + t)m \\ a_1 a_2 = (b_1 b_2) + (b_1 t + b_2 s + stm)m \end{cases}.$$

Hence the result. □

Corollary 3.9.1

Let $m, k \in \mathbb{N}^*$ and $a, b \in \mathbb{Z}$. If $a \equiv b (\mathrm{mod}\, m)$, then $a^k \equiv b^k (\mathrm{mod}\, m)$.

Also since $a \equiv (a \bmod m) (\mathrm{mod}\, m)$, we get that:

Corollary 3.9.2

Let $a, b \in \mathbb{Z}$ and $m \in \mathbb{N}^*$. Then

$$\begin{cases} (a+b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m, \\ (a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m. \end{cases}$$

Important

Some operations may not go as expected! For instance, while

$$a \equiv b (\bmod m) \Rightarrow \forall c \in \mathbb{Z}, a \cdot c \equiv b \cdot c (\bmod m),$$

the converse is not necessarily true, meaning:

$$a \cdot c \equiv b \cdot c (\bmod m) \not\Rightarrow a \equiv b (\bmod m).$$

For instance: can you divide both sides of the equivalence by 3 in $12 \equiv 6 (\bmod 3)$?

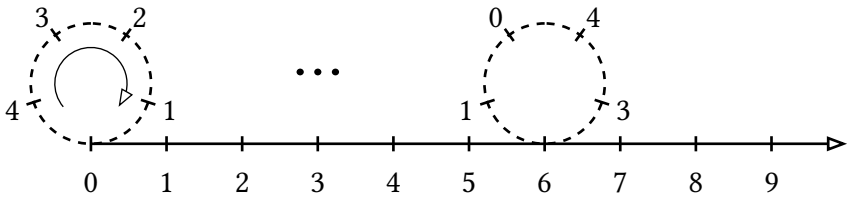
3.2.2 Arithmetic modulo m

To study operations involving congruences properly, let's first see how they define particular sets of integers with their own arithmetic.

Theorem 3.10

For each positive integer m, the binary relation $\equiv (\bmod m)$ is an equivalence relation, as defined in Section 2.2.

A good way to visualize this relation is to imagine that the straight line of integers gets rolled into a wheel with m ticks:



This is equivalent to defining the quotient set we'll introduce below!

Definition 3.11 (Congruence classes)

We define the congruence classes modulo m as

$$[r]_m = \{b \in \mathbb{Z} \mid b \equiv r (\bmod m)\} = \{q \cdot m + r \mid q \in \mathbb{Z}\}$$

It follows from Theorem 3.10 that congruence classes are equivalence classes. Thus, to a given modulo m correspond m distinct equivalence classes. And from Theorem 2.11 follows that:

Theorem 3.12

For any positive integer m, the congruence classes modulo m form a partition of $\mathbb{Z}$.

We can then define the set of the "simplest" representatives of these congruence classes: those corresponding to a quotient q equal to zero.

Definition 3.13

For any positive integer m, we define

$$\mathbb{Z}_m = \{r \in \mathbb{Z} \mid 0 \leq r \leq m-1\} = \{0, 1, ..., m-1\}$$

We know that any congruence modulo m can be performed the same way on these representatives as on all members of their respective classes: we can thus define an **arithmetic modulo m** on $\mathbb{Z}_m$.

Arithmetic on $\mathbb{Z}_m$, however, can be distinct from the one you know on $\mathbb{Z}$. Still, we know from Corollary 3.9.2 that they share similarities, which allows us to provide the following definitions.

Definition 3.14

For a given $m \in \mathbb{N}^*$ and $x, y \in \mathbb{Z}_m$, we define:

- the addition $+_m$:

$$x +_m y = (x + y) \bmod m,$$

- and the multiplication $\cdot_m$:

$$x \cdot_m y = (x \cdot y) \bmod m,$$

where the operations on the right-hand side are the ones on $\mathbb{Z}$.

These operations satisfy many of the familiar properties that addition and multiplication have on $\mathbb{Z}$. That is, for any $x, y \in \mathbb{Z}_m$:

- Closure: $x +_m y \in \mathbb{Z}_m$ and $x \cdot_m y \in \mathbb{Z}_m$.
- Associativity: $x +_m (y +_m z) = (x +_m y) +_m z$ and $x \cdot_m (y \cdot_m z) = (x \cdot_m y) \cdot_m z$.
- Commutativity: $x +_m y = y +_m x$ and $x \cdot_m y = y \cdot_m x$.
- Distributivity: $x \cdot_m (y +_m z) = x \cdot_m y +_m x \cdot_m z$.
- Identity element (sum): $\exists 0_m \in \mathbb{Z}_m$ such that $0_m +_m x = x$.
- Identity element (product): $\exists 1_m \in \mathbb{Z}_m$ such that $1_m \cdot_m x = x$.
- Additive inverses: $\exists (-_m x) \in \mathbb{Z}_m$ such that $x +_m (-_m x) = 0_m$.

Check at home

Check they actually satisfy them! What are 0_m, 1_m and $(-x_m)$ with regards to their counterparts in $\mathbb{Z}$?

Multiplicative inverses do not appear here, simply because they do not always exist in $\mathbb{Z}_m$. For instance, there is no multiplicative inverse of 2 modulo 6, as you can verify.

Definition 3.15 (Multiplicative inverse)

An element $r \in \mathbb{Z}_m$ has a multiplicative inverse modulo m if there is an element $s \in \mathbb{Z}_m$ such that $r \cdot s \equiv 1 (\mathrm{mod}\, m)$.

We will return to multiplicative inverses later on.

3.3 Greatest common divisors and least common multiples

3.3.1 Definitions

Definition 3.16 (Greatest common divisor)

Let a, b be integers which are not both zero. The largest positive integer d that divides both a and b is called the **greatest common divisor** of a and b. It is denoted by $\gcd(a, b)$.

Remark

- The case $a = b = 0$ is excluded because any integer divides 0.
- $\gcd(0, a) = |a|$ for all nonzero integers a.
- $\gcd(a, b) = \gcd(|a|, |b|)$

Definition 3.17 (Relative primes)

Two integers a and b are **relatively prime** if $\gcd(a, b) = 1$.

The integers $a_1, a_2, \ldots, a_n$ are pairwise relatively prime if $\gcd(a_i, a_j) = 1$ for any $1 \leq i < j \leq n$.

Definition 3.18 (Least common multiple)

The **least common multiple** of two natural numbers a, b is the smallest natural number m that is divisible by both a and b, that is $a \mid m$ and $b \mid m$. It is denoted by $\mathrm{lcm}(a, b)$.

3.3.2 How to compute them

The gcd can be obtained systematically using the following lemma.

Lemma 3.19 (Euclid's lemma)

Given the integers a, $b \neq 0$, q and r, such that $a = q \cdot b + r$ with $0 \leq r < |b|$, then $\gcd(a, b) = \gcd(b, r)$.

But how? Let's write $r_0 = |a|$, $r_1 = |b|$, $r_2 = r$ and $q_1 = q$ and apply the lemma successively:

$$\begin{aligned}
r_0 &= q_1 r_1 + r_2 && 0 \leq r_2 < r_1, \\
r_1 &= q_2 r_2 + r_3 && 0 \leq r_3 < r_2, \\
&\vdots \\
r_{n-2} &= q_{n-1} r_{n-1} + r_n && 0 \leq r_n < r_{n-1}, \\
r_{n-1} &= q_n r_n.
\end{aligned}$$

Since we have by Theorem 3.4 that $|a| = r_0 > r_1 > ... > r_n \geq 0$, this sequence of remainders cannot contain more than $|a|$ terms, so it has a finite size n. This means that there exists an n such that $r_n \mid r_{n-1}$.

It then follows from Lemma 3.19 that

$$\begin{aligned}\gcd(a,b) &= \gcd(|a|,|b|) = \gcd(r_0,r_1) = \gcd(r_1,r_2) = ... \\ &= \gcd(r_{n-1},r_n) \\ &= r_n\end{aligned}$$

Example. Apply recursively Euclid's lemma to compute $\gcd(662, 414)$.

The lcm is then straightforward to compute since we can use the following theorem.

Theorem 3.20

If a, b are two natural numbers, then

$$\gcd(a,b) \cdot \operatorname{lcm}(a,b) = a \cdot b$$

3.3.3 How to use them in modular arithmetic

Theorem 3.21 (Bézout's Identity)

If a and b are two integers not simultaneously zero, then their gcd can be written as a linear combination of these two integers. In other words:

$$\exists (x,y) \in \mathbb{Z}^2, \gcd(a,b) = a \cdot x + b \cdot y$$

Proof: We write the steps of Euclid's algorithm, and "unroll them":

$$\begin{aligned}a &= q_1 \cdot b + r_2 \Rightarrow r_2 = a - q_1 \cdot b \\ b &= q_2 \cdot r_2 + r_3 \Rightarrow r_3 = b - q_2 \cdot r_2 \\ r_2 &= q_3 \cdot r_3 + r_4 \Rightarrow r_4 = r_2 - q_3 \cdot r_3 \\ &\vdots \\ r_{n-3} &= q_{n-2} \cdot r_{n-2} + r_{n-1} \Rightarrow r_{n-1} = r_{n-3} - q_{n-2} \cdot r_{n-2} \\ r_{n-2} &= q_{n-1} \cdot r_{n-1} + r_n \Rightarrow r_n = r_{n-2} - q_{n-1} \cdot r_{n-1} \\ r_{n-1} &= q_n \cdot r_n \Rightarrow r_n = \gcd(a,b)\end{aligned}$$

Then,

$$\begin{aligned}\gcd(a,b) = r_n &= \alpha_{n-1} r_{n-2} + \beta_{n-1} r_{n-1} \\ &= \alpha_{n-2} r_{n-3} + \beta_{n-2} r_{n-2} \\ &= ... \\ &= \alpha_3 r_2 + \beta_3 r_3 \\ &= \alpha_2 b + \beta_2 r_2 \\ &= \alpha_1 a + \beta_1 b\end{aligned}$$

□

Remark

Bézout's identity does not imply that the integers x and y are unique.

The identity allows us to redefine the gcd through the following theorem.

Theorem 3.22

Let a and b be two integers not simultaneously zero with $\gcd(a, b) = d$.

An integer c can be written in the form $a \cdot x + b \cdot y$ for some integers x and y if and only if c is a multiple of d.

In particular, the gcd d is the smallest positive integer of the form $a \cdot x + b \cdot y$ with $x, y \in \mathbb{Z}$.

And as a direct consequence, to redefine relatively-prime integers through the following corollary.

Corollary 3.22.1

Two integers are relatively prime if and only if there exist integers x and y such that

$$a \cdot x + b \cdot y = 1$$

Corollary 3.22.2

If $\gcd(a, b) = d$, then

(i) $$\forall m \in \mathbb{N}, \gcd(m \cdot a, m \cdot b) = m \cdot d$$

(ii) $$\gcd\left(\frac{a}{d}, \frac{b}{d}\right) = 1$$

Corollary 3.22.3

If a, b are two relatively-prime integers, then:

(i) If $a \mid c$ and $b \mid c$, then $(a \cdot b) \mid c$.

(ii) If $a \mid (b \cdot c)$, then $a \mid c$.

Theorem 3.23

Let a, b, c be integers, and m a positive integer relatively prime to c. Then

$$a \cdot c \equiv b \cdot c (\mathrm{mod}\, m) \Rightarrow a \equiv b (\mathrm{mod}\, m).$$

Remark

- This theorem allows us to divide by a common factor c both sides of the sign $\equiv$ whenever c and the modulus m are relatively prime.

- If c and m are not relatively prime, then the correct result is:
  Let us write $m = p \cdot c$ for positive integers p, c, and let a, b be integers. If $a \cdot c \equiv b \cdot c (\mathrm{mod}\, p \cdot c)$, then $a \equiv b (\mathrm{mod}\, p)$.

3.4 Prime numbers

3.4.1 Definition

Definition 3.24 (Prime numbers)

A natural number $p > 1$ is called a **prime number** if its only positive divisors are 1 and itself.

A natural number $p > 1$ that is not prime is called **composite**.

Remark

The natural number 1 is not prime. The first prime number is 2, and the other prime numbers are odd natural numbers $(3, 5, 7, 11, \ldots)$.

Theorem 3.25 (Euclid)

There are infinitely many prime numbers.

Check at home

How primes are distributed is a topic that has attracted interest and fascination for centuries. You can check out this cool video on prime numbers (and this cool channel in general!) for an example of how it can connect to other problems.

3.4.2 Primes as building blocks

Since a prime only has 1 and itself as divisors, the gcd of a prime and any other number can only be 1 or the prime. Thus:

Lemma 3.26

Let p be a prime number, and let a be an integer. Then, either $p \mid a$, or p and a are relatively prime.

And using Corollary 3.22.3, we can then get that:

Lemma 3.27

If p is prime and $p \mid \prod_{i=1}^{n} a_i$, where each a_i is an integer, then $p \mid a_i$ for some i.

which leads us to a crucial result...

Theorem 3.28 (The Fundamental Theorem of Arithmetic)

Every natural number $n > 1$ can be written uniquely as a product of primes

$$n = p_1^{n_1} \cdot p_2^{n_2} \cdot p_3^{n_3} \cdot \ldots \cdot p_k^{n_k}$$

where the p_i are distinct prime numbers written in increasing order, and the exponents n_i are natural numbers $n_i \geq 1$.

Proof:

Existence of the factorization This can be proven by induction.

Basis step For $n = 2$, the factorization exists since 2 itself is prime.

Inductive step Let us assume that for a given n, all integers $2 \leq j \leq n$ have a prime factorization.

- First, if $n + 1$ is prime, its prime factorization is simply itself, hence its existence.
- If it is composite, it means it can be written as $n + 1 = a \cdot b$, where $2 \leq a \leq b \leq n$. By the inductive hypothesis, both a and b have prime factorizations, so $n + 1$ can also be factorized, by combining these two.

We thus just proved the existence of the factorization for any $n \geq 2$.

Uniqueness This can be proven by contradiction. Let's suppose that $n \in \mathbb{N}$ admits two distinct factorizations, so there exist $s \geq 1$ primes p_i and $t \geq 1$ primes q_j such that

$$n = \prod_{i=1}^{s} p_i = \prod_{j=1}^{t} q_j,$$

and such that we can simplify the equality above by the common factors, and get

$$\prod_{k=1}^{u} p_{i_k} = \prod_{l=1}^{v} q_{j_l},$$

with $u, v \geq 1$. For each k, we know that $p_{i_k} \mid n$, and so, by Lemma 3.27, necessarily $p_{i_k} \mid q_{j_l}$ for some l. Since these numbers are primes, this is impossible, by definition. Hence the uniqueness of the factorization.

□

Remark

If you're curious as to why this theorem is *fundamental*, you can check out this nice video.

This theorem first allows us to prove another that makes it easier to determine if an integer is prime.

Theorem 3.29

The positive integer n is a composite number if and only if n can be divided by some prime number $p \leq \sqrt{n}$.

Example. See Exercise 3.7

The prime factorization also helps obtaining the gcd and lcm of two integers.

Proposition 3.30

If the integers $a, b > 1$ can be factorized in the form

$$a = p_1^{n_1} \cdot p_2^{n_2} ... p_k^{n_k}$$
$$b = p_1^{m_1} \cdot p_2^{m_2} ... p_k^{m_k}$$

with $n_i, m_i \geq 0$, then

$$\gcd(a,b) = p_1^{\min(n_1,m_1)} \cdot p_2^{\min(n_2,m_2)} \cdot ... \cdot p_k^{\min(n_k,m_k)}$$
$$\operatorname{lcm}(a,b) = p_1^{\max(n_1,m_1)} \cdot p_2^{\max(n_2,m_2)} \cdot ... \cdot p_k^{\max(n_k,m_k)}$$

3.5 Solving congruences

3.5.1 Congruences involving very large numbers

Results on primes can also be used to compute the remainder of divisions of some very high numbers, even when they can't be stored in a computer's memory. How? Using the following.

Theorem 3.31 (Fermat's little theorem)

Let p be a prime number and y any integer. Then, if $y \not\equiv 0 (\operatorname{mod} p)$, we have

$$y^{p-1} \equiv 1 (\operatorname{mod} p)$$

Proof: See exercises. □

Corollary 3.31.1

If p is a prime number, then $y^p \equiv y (\operatorname{mod} p)$ for any integer y.

But what if the divisor is not prime? Then, Theorem 3.28 comes to the rescue, paired with the following.

Theorem 3.32 (Chinese remainder theorem)

Let $m_1, m_2, ..., m_n \in \mathbb{N} \setminus \{0, 1\}$ be pairwise relative primes, and $a_1, a_2, ..., a_n \in \mathbb{N}$. Then the system

$$\begin{cases} x \equiv a_1 (\operatorname{mod} m_1) \\ x \equiv a_2 (\operatorname{mod} m_2) \\ \quad \vdots \\ x \equiv a_n (\operatorname{mod} m_n) \end{cases}$$

has a unique solution modulo $m = \prod_{i=1}^{n} m_i$.

Then, to solve $y^n \equiv x (\operatorname{mod} m)$, we factor m into primes m_i, and then solve the system from Theorem 3.32.

3.5.2 Linear congruence equations

Being able to solve linear equations is fundamental to many problems of calculus or linear algebra, and the same is true for linear *congruence* equations in number theory.

Definition 3.33

A linear congruence equation is a congruence modulo m of the form

$$a \cdot x \equiv b (\mathrm{mod}\, m)$$

where m is a positive integer, a, b are integers, and x is a variable.

To solve these equations, we can use multiplicative inverses (from Definition 3.15). Indeed, let's consider $m > 1$, and assume a admits a multiplicative inverse $a^{-1} \in \mathbb{Z}_m$. Then, we can multiply both sides of the linear congruence equation by a^{-1} to isolate x:

$$a \cdot x \equiv b (\mathrm{mod}\, m) \Rightarrow x \equiv a^{-1} \cdot b (\mathrm{mod}\, m).$$

The solutions x are then all integers congruent to $a^{-1} \cdot b$ modulo m. But let's first see when we can get such an inverse.

Theorem 3.34

An element $r \in \mathbb{Z}_m$ has a multiplicative inverse if and only if r and m are relatively prime.

Corollary 3.34.1

If p is a prime number, every nonzero element of $\mathbb{Z}_p$ is invertible.

Here's why we stopped right after introducing multiplicative inverses above: we needed to introduce relatively prime numbers before being able to condition their existence.

Theorem 3.35

For any $r \in \mathbb{Z}_m$, if its multiplicative inverse exists, then it is unique modulo m.

Notation

Since the inverse of r modulo m is unique when it exists, it will be denoted by r^{-1}.

Remark

For small values of m finding a multiplicative inverse is straightforward: we can simply multiply a by $2, \ldots, m-1$ until the result exceeds a multiple of m by 1.

For higher m, we can rather use Bézout (Theorem 3.21) to find coefficients x and y such that

$$\gcd(a, m) = 1 = a \cdot x + m \cdot y \Leftrightarrow a \cdot x \equiv 1 (\mathrm{mod}\, m)$$

It will then follow that x is an inverse of a modulo m.

We can thus provide the following way to solve linear congruence equations.

Theorem 3.36

If a and m are relatively prime integers, then the linear congruence equation

$$a \cdot x \equiv b (\mathrm{mod}\, m)$$

has a unique solution modulo m, which is $x_0 = (a^{-1} \cdot b) \bmod m$, where a^{-1} is the inverse of a modulo m.

The general solution is then given by

$$\forall k \in \mathbb{Z}, x_k = x_0 + (k \cdot m),$$

which forms a congruent class modulo m.

3.6 Applications

3.6.1 Random number generators

Many algorithms need to draw random numbers to work, for instance in statistical sampling, cryptography or to train machine-learning models. Because generating actually-random numbers needs a natural source of randomness, such as complex atmospheric phenomena, it is limited by the "rate of randomness" that these sources can provide. Most often we thus rather generate *pseudorandom* numbers in computers.

A popular procedure to do so is called the **linear congruential method**. It consists in first choosing four integers:

- the modulus m (large!),
- the multiplier $a \in [\![2, m-1]\!]$,
- the increment $c \in [\![0, m-1]\!]$,
- the seed $x_0 \in [\![0, m-1]\!]$.

Using these, a sequence of pseudorandom numbers can be generated by successively applying

$$x_{n+1} = (a x_n + c) \bmod m.$$

3.6.2 Cryptography

Number theory, and particularly modular arithmetic, is the basis for many cryptography techniques. Perhaps the simplest example is Caesar's encryption process. It maps each letter of the latin alphabet to a number in $\mathbb{Z}_{26}$, and shifts them by an arbitrary k modulo 26. It can thus be written as the function

$$f : \mathbb{Z}_{26} \to \mathbb{Z}_{26}, f(l) = (l + k) \bmod 26.$$

Check at home

More complex techniques which are actually widely-used nowadays also involve modular arithmetic. One example is the RSA system, which is based on Fermat's little theorem (Theorem 3.31). For more information, see section 4.6 of Rosen.

4 Counting

This chapter overlaps with sections 6.1, 6.2, 6.3, 6.4, 6.5 of Rosen.

The **goal of counting** is basically to determine the cardinality of certain finite sets. This is much more useful than it sounds. For instance, a fundamental way to compute discrete probabilities is to count in how many ways a given event can occur, and divide it by the total number of possible alternatives.

Definition 4.1

Two sets A and B have the same cardinality if and only if there exists a bijective function $f : A \to B$.

Definition 4.2 (Countable sets)

A set that is either finite or has the same cardinality as the set $\mathbb{N}$ is called **countable**.

4.1 Fundamental counting principles

4.1.1 The product rule

Suppose that a meal can be broken down into a first and second course. Let's assume we have n_1 recipes for the first course and n_2 for the second. If any combination of first and second course is deemed acceptable, then we have $n_1 \cdot n_2$ ways to prepare a whole meal.

Remark

The condition above can be rephrased as "if the choice of second course is independent from the choice of first course". "Independent" is a word you'll hear again in probability theory. This is no coincidence!

The recipes for the first and second course can be seen as the members of two sets, which can be combined together through the Cartesian product of these two sets. This is the idea behind the product rule stated below.

Proposition 4.3 (The basic product rule)

If A and B are two finite sets, then

$$|A \times B| = |A| \cdot |B|$$

This can be directly generalised to an arbitrary number of sets.

Proposition 4.4 (The generalised product rule)

If $A_1, A_2, ..., A_m$ are finite sets, then

$$|A_1 \times A_2 \times ... \times A_m| = |A_1| \cdot |A_2| \cdot ... \cdot |A_m| = \prod_{k=1}^{m} |A_k|$$

Example. How many functions are there from a set with m elements to a set with n elements?

4.1.2 The division rule

Still with our recipes, imagine that to prepare the same meal, you have N different recipes, but for each of those, you find $d-1$ others which taste the same. Then the number of actually different recipes you can taste is $\frac{N}{d}$. In terms of sets, this gives

Proposition 4.5 (The division rule)

If a finite set A is the union of n pairwise disjoint subsets, each of cardinality d, then $|A| = n \cdot d$.

which is called the division rule, because most often, the unknown is $n = \frac{|A|}{d}$, as we'll see later on.

Example. How many different ways are there to seat four people around a circular table, where two seatings are considered the same when each person has the same neighbors?

4.1.3 The sum rule

Let's consider the cuisines of two different countries. Let's assume a given ingredient can be prepared in n_1 recipes in the first, and in n_2 recipes in the second. If the two countries do not share a single recipe in common, then someone who knows both cuisines can prepare $n_1 + n_2$ different recipes with this ingredient.

Proposition 4.6 (The basic sum rule)

If A and B are two finite and disjoint sets ($A \cap B = \emptyset$), then

$$|A \cup B| = |A| + |B|$$

This can be directly generalised to an arbitrary number of sets.

Proposition 4.7 (The generalised sum rule)

If $A_1, A_2, ..., A_m$ are a sequence of finite and pairwise disjoint sets $A_i \cap A_j = \emptyset$ for all $i \neq j$, then

$$\left|\bigcup_{i=1}^{m} A_i\right| = |A_1 \cup A_2 \cup ... \cup A_m| = |A_1| + |A_2| + ... + |A_m| = \sum_{i=1}^{m} |A_i|$$

4.1.4 The principle of inclusion-exclusion

We can generalise the sum rule by relaxing the assumption we made that the two cuisines do not share a single recipe in common. So let's be more agnostic and consider the possibility that they might. This leads to the principle of inclusion-exclusion.

Proposition 4.8 (The basic principle of inclusion-exclusion)

If A and B are two finite sets, then

$$|A \cup B| = |A| + |B| - |A \cap B|$$

You may see this graphically by drawing a Venn diagram.

Question

Now, what happens if we add another set C to the mix? Try to answer by drawing another diagram.

The generalisation is less obvious than before, but as hinted by our iterative approach of adding one subset at a time, by induction we get

Proposition 4.9 (The generalised principle of inclusion-exclusion)

If $A_1, A_2, ..., A_m$ are finite sets, then

$$\left|\bigcup_{i=1}^{m} A_i\right| = \sum_{1\le i\le m} |A_i| - \sum_{1\le i<j\le m} |A_i \cap A_j| + \sum_{1\le i<j<k\le m} |A_i \cap A_j \cap A_k| \\ ... + (-1)^{m+1}|A_1 \cap A_2 \cap ... \cap A_m|$$

4.1.5 The pigeonhole principle

Imagine that n pigeons try to land into $n-1$ pigeonholes. Clearly, there are not enough pigeonholes for every pigeon to have its own, which implies that at least two of them will share a pigeonhole. In terms of objects and boxes, this gives

Proposition 4.10 (The pigeonhole principle)

If $k+1$ or more objects are placed into k boxes, then there is at least one box containing two or more of the objects.

Apart from its funny name, the pigeonhole principle has useful implications, especially when generalised.

Proposition 4.11 (The generalised pigeonhole principle)

If N or more objects are placed into k boxes, then there is at least one box containing at least $\lceil N \,/\, k \rceil$ objects.

Proof: Suppose that none of the boxes contains more than $\lceil N \,/\, k \rceil - 1$ objects. Then, the total number of objects is at most

$$k\left(\left\lceil \frac{N}{k} \right\rceil - 1\right) < k\left(\left(\frac{N}{k} + 1\right) - 1\right) = N.$$

This proves the principle by contraposition. □

Example. Among 100 people, how many are born in the same month?

Example. How many cards must be selected from a standard deck of 52 cards to guarantee that at least three cards of the same suit are selected?

4.2 Combinatorics

4.2.1 Permutations

A permutation is basically a way in which elements of a set can be ordered.

Example. Let's consider the set containing the first four letters of the alphabet: $S = \{A, B, C, D\}$. In how many ways can we order them, or, in other words, in how many ways can we permute the elements of this set, or, even, how many permutations does this set have?

Solution. To see how to answer that, let's think about the very process of ordering. We first choose the element that should come first: at this stage we have 4 different choices. Once this choice is made, we have 3 choices left for the second, then after that, 2 choices for the third, and finally, only one. By the product rule we thus have $4 \cdot 3 \cdot 2 \cdot 1 = 24$ ways to order these elements.

Proposition 4.12 (Permutations of n distinct objects)

Given n distinct objects, there are $n!$ distinct ordered arrangements (= permutations) of these objects.

Remark

For a set A, we can directly say that it has $|A|!$ permutations, since its elements are distinct by definition.

We just involved the factorial, that we now properly define.

Definition 4.13 (Factorial)

For all $n \in \mathbb{N}$, we define the **factorial** of n as

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 2 \cdot 1 = \prod_{k=1}^{n} k$$

Remark

By convention, $0! = 1$.

Let us generalise the concept of a permutation by also considering we want to form orderings of size $r \leq n$.

Definition 4.14 (Permutation)

Given a set of cardinality n and a natural number $r \leq n$, an ***r*-permutation** of elements of this set is an ordered arrangement of r elements of this set.

Notation

The number of such arrangements is denoted $P(n, r)$.

The result above can then be directly extended, as we simply stop making choices after r steps.

Proposition 4.15

Given a set of cardinality n and a natural number $r \leq n$, the number of r-permutations we can form from this set is:

$$P(n,r) = n \cdot (n-1) \cdot \ldots \cdot (n-r+1) = \frac{n!}{(n-r)!}$$

Remark

We can get back the result of Proposition 4.12 for n-permutations using the convention $0! = 1$.

If we tweak the permutation process by considering that each choice is independent from the others, we're now allowed to pick any element from the set at every step, even those we've already picked before. We're thus allowing repetitions in our permutation, and the number of such permutations is even more straightforward to obtain.

Proposition 4.16

The number of r-permutations of a set of n objects with repetition allowed is n^r.

Now what happens if our objects are not necessarily distinguishable anymore? Let's say we now want to know in how many ways we can the shuffle the string $ABBCCC$.

Solution. If we attached labels to each letter to keep track of them, like such: $A_1B_1B_2C_1C_2C_3$, then as above, we would have $(1+2+3)! = 6!$ permutations. But notice that if we remove the labels, the two following permutations

$$A_1B_1B_2C_1C_2C_3$$
$$A_1B_2B_1C_1C_2C_3$$

in which we permuted the B's, would look exactly the same! So for each labeled permutation, two unlabeled ones would have the same ordering for the B's. Similarly, the C's can be reordered in 3! ways in each labeled permutation.

Therefore, all in all, each labeled permutation has $2! \cdot 3! = 12$ equivalent unlabeled ones. We can thus get the number of ways we can shuffle this string as:

$$\frac{6!}{2! \cdot 3!} = \frac{6 \cdot 5 \cdot 4}{2} = 60$$

Proposition 4.17 (Permutations with indistinguishable objects)

Given n objects that can classified into k groups of indistinguishable elements, and such that the first group contains n_1 indistinguishable elements of type 1, the second group contains n_2 indistinguishable elements of type 2, etc, then the number of distinct ordered arrangements of these objects is

$$\frac{n!}{n_1! \cdot n_2! \cdot \ldots \cdot n_k!}$$

Remark

By definition of the n_i above, $\sum_{i=1}^{k} n_i = n$.

4.2.2 Combinations

What if we now count distinct unordered selections of objects from a set, that is, how many distinct subsets of the same size we can form?

This is called a combination. If we simply combine all the elements from a set with n elements, by definition of a set there is only one such combination of n elements. What's interesting is if we wonder how many combinations of $r < n$ elements from this set of n elements we can form.

Definition 4.18 (Combination)

Given a set of cardinality n and $r \leq n$, an ***r*-combination** of elements of this set is one of its subsets of cardinality r.

Notation

The number of such subsets is denoted $C(n, r)$.

As we've seen above, a set of size n admits $\frac{n!}{(n-r)!}$ permutations of size r. But, once we have an r -permutation, this new arrangement itself can be permuted in $P(r, r) = r!$ ways, each of which corresponds to the same subset. That's how we can see by the division rule that

Proposition 4.19

Given a set of cardinality n and a natural number $r \leq n$, the number of r-combinations we can form from this set is:

$$C(n, r) = \frac{P(n, r)}{r!} = \frac{n!}{r!(n-r)!}$$

Remark

Notice how this does give us $C(n, n) = 1$, using again the convention $0! = 1$.

This leads us to introduce some particular integers: the binomial coefficients.

Definition 4.20 (Binomial coefficients)

For all non-negative integers $n, r \in \mathbb{Z}^+$ such that $0 \leq r \leq n$, we define the **binomial coefficient** as follows:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

Question

Rewrite $C(n, r)$ and $P(n, r)$ using a binomial coefficient.

Remark

The symbol $\binom{n}{r}$ is read "n choose r", as it gives us the number of ways we can choose r distinct elements from a set of size n. Forming an r-combination is thus equivalent to sampling without replacement, to translate our combinatorics approach to the language of statistics.

4.2.3 Distributions

Many counting problems can be solved by mapping them to a process of distributing objects into boxes. Same as above, the objects can be considered distinguishable ($\sim$ labeled), or not. The distinction could also be made for the boxes, but in this course we will only consider distinguishable boxes.

Let's first consider n distinguishable objects to be placed into k boxes through an example.

Example. How many ways are there to distribute hands of 5 cards to each of four players from a deck of 52 cards?

Solution. This problem does correspond to the general one stated above, because all cards and players are distinct. We can notice that once we dealt cards to a player i, it is as if we created a group, or box, of $n_i = 5$ cards. We thus create one of these groups for each of the four players, and a final group with the remaining $52 - 4 \cdot 5 = 32$ cards. This amounts to creating $k = 5$ groups, within each of which we can shuffle the cards without changing the distribution.

We can thus form a bijective function between the distributions of cards to the players and permutations with indistinguishable objects from Proposition 4.17! From Definition 4.1, we can therefore use the previous result to get that the total number of ways to distribute the hands is

$$\frac{52!}{(5!)^4 \cdot 32!}$$

We thus get the same result as in Proposition 4.17.

Proposition 4.21 (Distributions of distinguishable objects)

The number of distributions of n distinguishable objects into k distinguishable boxes so that each box i receives n_i objects is

$$\frac{n!}{n_1! \cdot n_2! \cdot \ldots \cdot n_k!}$$

Let's now consider we have n indistinguishable objects.

- Since objects are indistinguishable, they can be represented by any same symbol, such as a star $*$, which is conventionally used as a placeholder.

- The assignment to k boxes can then be represented by separating these stars with $k-1$ bars.

We thus use a **"stars and bars"** representation, which is very commonly used to solve this kind of problems.

Example. For $n = 8$ and $k = 3$, two example distributions can thus be visualised as:

$$\underbrace{***}_{A} \mid \underbrace{****}_{B} \mid \underbrace{*}_{C} \qquad \underbrace{\ }_{A} \mid \underbrace{****}_{B} \mid \underbrace{****}_{C}$$

So the $n + k - 1 = 10$ stars and bars are to be arranged in as many slots, so, for instance, we can first choose n of these spots for our n stars, and fill the remaining ones with bars.

This is thus a "$(n + k - 1)$ choose n" combination!

Proposition 4.22 (Distributions of indistinguishable objects)

The number of distributions of n indistinguishable objects into k distinguishable boxes is

$$\binom{n+k-1}{n}$$

Another parallel with combinations can be established. Counting the number of ways of placing n indistinguishable objects into k boxes turns out to be the same as counting the number of n-combinations for a set with k elements when repetitions are allowed. These are slightly different from Definition 4.18, since we allow to redraw the same element from the input set. The combination with repetition is thus not a subset, but a collection of elements with potential repetitions. The processes behind the distribution and the combination with repetition can be directly mapped by imagining that each time the i^{th} element of the input set is included in the combination, we put a ball in the i^{th} box of the distribution.

Proposition 4.23 (r-combinations with repetition)

Given a set of cardinality n and a natural number r, the number of r-combinations with repetition that we can form from this set is:

$$\binom{r+n-1}{n-1}$$

4.2.4 Binomial coefficients

There are many useful identities related to these coefficients. One of the most important is Pascal's.

Theorem 4.24 (Pascal's identity)

$$\forall r \in \mathbb{Z}^+, n \geq r, \binom{n+1}{r} = \binom{n}{r} + \binom{n}{r-1}$$

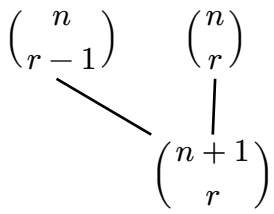
Using this identity, the binomial coefficients can be visualised by arranging them such that

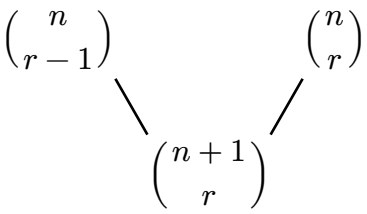
$$\binom{n}{r} \equiv \binom{\text{row number}}{\text{column number}}$$

Starting with the initial conditions $\binom{n}{0} = \binom{n}{n} = 1$, we thus form Pascal's triangle, expanding it downwards using Theorem 4.24. Pascal's triangle can then be drawn left or centre-aligned, as preferred:

1											
1	1										
1	2	1									
1	3	3	1								
1	4	6	4	1							
1	5	10	10	5	1						

					1						
				1		1					
			1		2		1				
		1		3		3		1			
	1		4		6		4		1		
1		5		10		10		5		1	





This also allows us to see the symmetry in the binomial coefficients, especially using the centre-aligned representation.

Theorem 4.25 (Symmetry)

$$\forall r \in \mathbb{N}, n \geq r, \binom{n}{r} = \binom{n}{n-r} = \frac{n!}{r!(n-r)!}$$

The binomial coefficients also appear in the expansion of some polynomials.

Example. Let's take the example of the expansion of $(x+y)^3$. Its expansion can be seen as a combinatorics problem in which we repeatedly pick either x or y from each parenthesised term of $(x+y)^3 = (x+y)(x+y)(x+y)$ to form a product.

This gives us 2^3 products of the form $x^\alpha y^\beta$ with $0 \leq \alpha, \beta \leq 3$, which we then sum. We can thus simply count the number of ways to choose α times x from the 3 parenthesised terms, which is $\binom{3}{\alpha}$! Then, $\beta = 3 - \alpha$. Explicitly, here:

$$(x+y)^3 = xxx + xxy + xyx + xyy + yxx + yxy + yyx + yyy,$$

so the number of ways we can put two x in three slots is $\binom{3}{2}$, to put only 1 is $\binom{3}{1}$ and only $\binom{3}{3} = \binom{3}{0} = 1$ way to put either three or zero of them.

This leads to the binomial theorem.

Theorem 4.26 (Newton's binomial theorem)

Let x and y be two variables, and $n \in \mathbb{N}$. Then

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}$$

Corollary 4.26.1

$$(1+x)^n = \sum_{k=0}^{n} \binom{n}{k} x^k$$

Corollary 4.26.2

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n, \quad \sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0$$

Finally, the binomial theorem together with Proposition 4.19 enables us the determine the cardinality of the power set.

Corollary 4.26.3

Given a finite set A of cardinality n,

$$|\mathcal{P}(A)| = \sum_{r=0}^{n} C(n,r) = 2^n$$

Remark

You will find again the binomial coefficients in the binomial distribution and other distributions in probability theory.

5 Sequences

This chapter overlaps with sections 2.4, 8.2, 8.4 of Rosen.

5.1 Defining sequences

Sequences are basically indexed collections of elements, or, to be more formal:

Definition 5.1 (Sequence)

A sequence is a function from a set of indices $I \subseteq \mathbb{N}$ to a set A of elements at each index. The image in A of the integer $n \in I$ is denoted a_n, and is called the n^{th} term of the sequence. The sequence can be represented as $(a_n)_{n \in I}$.

Remark

There are many ways to represent sequences in computers. In Python for instance, there are `bytes` (bytes sequences), `str` (character sequences), `list`, `tuple`, or even `range` which are sequence types. Note how even in Python the difference is made between sequences and sets: the `Sequence`'s central method is `__getitem__` –which allows you to retrieve their nth element–, which doesn't exist for `Set`, whose central method is `__contains__`, to check if an element is inside the set.

The terms of a sequence can be defined explicitly, with a closed form...

Example. Let's define the sequence $(a_n)_{n \in \mathbb{N}}$ where $a_n = 2^n$. Then the terms of the sequence are $1, 2, 4, 8, \ldots$

... but not only! They may also be defined recursively, that is, providing some initial terms together with a rule that is needed to determine subsequent terms.

Definition 5.2 (Recurrence relation)

A **recurrence relation** for the sequence $(a_n)_{n \in \mathbb{N}}$ is an equation that expresses a_n in terms of one or more previous terms $a_{n-1}, a_{n-2}, \ldots, a_{n-k}$, and possibly n. The **initial conditions** are the first k terms in the sequence: $a_0, \ldots, a_{k-1}$; which are necessary to be able to determine all terms of such a sequence.

Example. Redefine the previous sequence ($a_n = 2^n$) and the factorial ($b_n = n!$) using recurrence relations.

Remark

- More than one sequence may satisfy the same recurrence relation, hence the importance of initial conditions.
- From the principle of strong induction (Theorem 2.24), we know that a recurrence relation together with a set of initial conditions define a unique sequence.

5.2 Aggregating sequences

Sometimes, we're not interested in every term of the sequence, but in some aggregate. For instance, you're not really interested in all movements in your bank account, but in your balance, so the sum of all these movements.

The two fundamental ways to aggregate a sequence are the summation and products, that we introduce below.

Definition 5.3 (Sequence sum)

Given a sequence $(a_n)_{n\in I}$ with $I \subseteq \mathbb{N}$, its **sum** is the sum of its terms. Very often, we sum over contiguous integers, so $I = \{m, m+1, ..., M\}$ with $M > m$, and we write it

$$\sum_{n\in I} a_n = a_m + a_{m+1} + ... + a_M = \sum_{n=m}^{M} a_n.$$

If I is infinite, the sum is called a **series**. If I is empty, the sum is 0.

Definition 5.4 (Sequence product)

Given a sequence $(a_n)_{n\in I}$ with $I \subseteq \mathbb{N}$, its **product** is the product of its terms. Very often, we sum over contiguous integers, so $I = \{m, m+1, ..., M\}$ with $M > m$, and we write it

$$\prod_{n\in I} a_n = a_m a_{m+1} ... a_M = \prod_{n=m}^{M} a_n.$$

If I is infinite, the sum is called an **infinite product**. If I is empty, the product is 1.

The usual laws of arithmetic apply to these sums and products.

Proposition 5.5

Let (x_n) and (y_n) be two sequences, $I \subseteq \mathbb{N}$ and $a, b \in \mathbb{R}$, then

$$\sum_{i\in I}(ax_n + by_n) = a\sum_{i\in I} x_n + b\sum_{i\in I} y_n$$

$$\prod_{i\in I}(x_n^a y_n^b) = \left(\prod_{i\in I} x_n\right)^a \left(\prod_{i\in I} y_n\right)^b$$

We can often prove a closed form for a sum or a product using induction. To show it, let's introduce two fundamental kinds of sequences.

Definition 5.6 (Arithmetic progression)

An arithmetic progression is a sequence $(a_n)_{n\in\mathbb{N}}$ that satisfies the recurrence relation

$$a_{n+1} = a_n + d$$

where $d \in \mathbb{R}$ is called the common difference.

Definition 5.7 (Geometric progression)

A geometric progression is a sequence $(a_n)_{n\in\mathbb{N}}$ that satisfies the recurrence relation

$$a_{n+1} = ra_n$$

where $r \in \mathbb{R}$ is called the common ratio.

Then the following can be proven by induction.

Proposition 5.8

For all $n \in \mathbb{N}$ and $r \in \mathbb{R} \setminus \{1\}$:

$$\text{(i)} \sum_{k=0}^{n} k = \frac{n(n+1)}{2}$$

$$\text{(ii)} \sum_{k=0}^{n} r^k = \frac{1-r^{n+1}}{1-r}$$

5.3 Solving recurrence relations

We have seen above how to translate an explicit sequence definition into a recurrence relation. However, most of the time, it's the other way around. In this case, we want to **solve** the recurrence relation, that is, to find the equivalent explicit sequence definition, also known as a **closed formula**, if it exists.

5.3.1 Linear homogeneous recurrence relations

Let us first study linear recurrence relations, as they often appear in real-world problems, and can be solved systematically.

Definition 5.9

- A recurrence relation is of $\boldsymbol{k^{\text{th}}}$ **order** if a_n can be expressed in terms of its k previous terms $a_{n-1}, a_{n-2}, \ldots, a_{n-k}$.
- A recurrence relation is **linear** if it expresses a_n as a linear combination of its previous terms.
- A recurrence relation is **homogeneous** if the zero sequence $a_n = 0$ satisfies the relation.

Proposition 5.10

A k^{th} order linear recurrence relation with constant coefficients is of the form

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k} + f(n)$$

where $c_1, \ldots, c_k \in \mathbb{R}$, $c_k \neq 0$, and $f : \mathbb{N} \to \mathbb{R}$ which does not depend on any sequence term. The relation is also homogeneous if and only if $f(n) = 0$ for all n.

To gain some intuition on the solution of such recurrence relations, let's start simple.

Theorem 5.11 (Solution of a homogeneous first-order recurrence relation)

Let us suppose that the sequence $(a_n)_{n\in\mathbb{N}}$ satisfies the recurrence relation

$$\forall n \geq 1, a_n = ca_{n-1},$$

where $c \in \mathbb{R}$, and we know the initial condition a_0. Then, the solution of this relation is given by

$$\forall n \in \mathbb{N}, a_n = a_0 c^n.$$

Let us now observe that $a_n = \alpha x^n$ with $\alpha, x \in \mathbb{R}^*$ is a solution of k^{th} order recurrence relation if and only if, for all $n \geq k$:

$$\begin{aligned} & \alpha x^n = c_1 \alpha x^{n-1} + c_2 \alpha x^{n-2} + \ldots + c_k \alpha x^{n-k} \\ \Leftrightarrow\ & x^k - c_1 x^{k-1} - c_2 x^{k-2} - \ldots - c_{k-1} x - c_k = 0 \\ \Leftrightarrow\ & P(x) = 0 \end{aligned} \tag{1}$$

where we introduced the **characteristic polynomial** P, whose roots we will call the **characteristic roots**.

The equivalence above implies that if x_i is a characteristic root, αx_i^n is a solution of the recurrence relation. The characteristic polynomial of a k^{th} order linear recurrence is of degree k, so it admits k characteristic roots: $x_1, \ldots, x_k$.

- Let us first suppose all roots are distinct. Then the solution is a linear combination of the form:

$$a_n = \sum_{i=1}^{k} \alpha_i x_i^n$$

- Let us now suppose we have a root x_i of multiplicity $m_i \geq 2$. Then the derivative P' of the characteristic polynomial also has x_i as a root, of multiplicity $m_i - 1$.

  Also, we can notice that Equation 1 is equivalent to $x^{n-k}P(x) = 0$, by retracing our steps. Now if we write $Q_1(x) = x^{n-k}P(x)$, then $Q_1'(x_i) = 0$, which we can rewrite as

$$nx_i^{n-1} = c_1(n-1)x_i^{n-2} + \ldots + c_k(n-k)x_i^{n-k-1}.$$

  Multiplying both side by x_i, we then see that $a_n = nx_i^n$ is a solution of the recurrence.

  Then, if $m_i \geq 3$, we can do the same with $Q_2(x) = xQ_1'(x)$ and find that n^2x^n is also a solution, and again with $m_i \geq 4$, etc.

  By induction, we can thus get that if x_i is a characteristic root of multiplicity m_i, then

$$nx_i^n,\ n^2x_i^n,\ \ldots,\ n^{m_i-1}x_i^n$$

  are also solutions of the recurrence.

Putting everything together, we get the following general result.

Theorem 5.12 (Solution of a linear homogeneous recurrence relation)

Let (a_n) be a sequence satisfying a recurrence relation of the form:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k}$$

with $c_1, \ldots, c_k \in \mathbb{R}$. Let us assume that its characteristic polynomial

$$P(x) = x^k - c_1 x^{k-1} - c_2 x^{k-2} - \ldots - c_{k-1} x - c_k$$

has r distinct roots $x_1, \ldots, x_r$ with multiplicities $m_1, \ldots, m_r$.

Then, a sequence (a_n) is a solution of the recurrence relation if and only if

$$\forall n \in \mathbb{N}, a_n = \sum_{i=1}^{r} \left(\sum_{j=1}^{m_i} \alpha_{i,j} n^{j-1} \right) x_i^n$$

where $\alpha_{i,j}$ are constants to be fixed according to initial conditions.

Let us see what this gives in a simple case, with Fibonacci-like recurrences.

Corollary 5.12.1 (Solution of a homogeneous Fibonacci-like recurrence relation)

Let us suppose that the sequence $(a_n)_{n\in\mathbb{N}}$ satisfies the recurrence relation

$$\forall n \geq 2, a_n = c_1 a_{n-1} + c_2 a_{n-2}$$

with $c_1, c_2 \in \mathbb{R}$. If the **characteristic equation** associated to this relation

$$x^2 = c_1 x + c_2$$

has characteristic roots x_1 and x_2, then the solution of the recurrence relation is

$$\forall n \in \mathbb{N}, a_n = \begin{cases} \alpha_1 x_1^n + \alpha_2 x_2^n & \text{if } x_1 \neq x_2 \\ (\alpha_1 + n\alpha_2) x_1^n & \text{if } x_1 = x_2 \end{cases}$$

where the constants α_1 and α_2 can be obtained using the initial conditions a_0, a_1.

5.3.2 Linear nonhomogeneous recurrence relation

What if we now add heterogeneity?

Theorem 5.13 (Solution of a linear nonhomogeneous recurrence relation)

Let us consider the linear nonhomogeneous recurrence relation with constant coefficients:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k} + f(n)$$

where $c_1, c_2, \ldots, c_k \in \mathbb{R}$, and the initial conditions $a_0, \ldots, a_{k-1}$ are known.

Then, the general solution of this linear nonhomogeneous recurrence is $\left(a_n^{(h)} + a_n^{(p)}\right)_{n\in\mathbb{N}}$, where $\left(a_n^{(h)}\right)$ is the solution of the associated *homogeneous* recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \ldots + c_k a_{n-k},$$

and $\left(a_n^{(p)}\right)$ is any *particular* solution of the nonhomogeneous relation.

So the hard part is to "guess" a particular solution of the nonhomogeneous relation. In some simple examples though, it is not too hard to make the right guess.

5.4 Generating functions

5.4.1 Definitions

We will now introduce generating functions, which are a very powerful tool to manipulate sequences. The idea is not to find a function which is "equal" to the sequence in any sense, but one that **encodes the sequence**.

To be more specific, when you formulate a generating function you compress all the information necessary to reproduce the full sequence, so the value of all its terms, into a simple function such as $f(x) = e^x$, $f(x) = \frac{1}{1-x}$, etc .

How can a function contain all this information, though? By using the fact that many functions can be written as a power series, which means there is a sequence $(a_n)_{n\in\mathbb{N}}$ such that

$$f(x) = \sum_{n=0}^{+\infty} a_n x^n.$$

An obvious example is the polynomials: for instance $P(x) = 1 + 2x^2$ can be written as a power series with sequence (a_n) such that $a_0 = 1, a_2 = 2,$ and $\forall n \in \mathbb{N} \setminus \{0, 2\}, a_n = 0$.

But many other functions can be written as a power series, for instance by using their Taylor expansion! For instance, $e^x = \sum_{n=0}^{+\infty} \frac{x^n}{n!}$, so its associated sequence is $a_n = \frac{1}{n!}$. We can thus say that e^x *generates* this sequence, in the sense that it can tell you the values of all its terms. Contrary to what you've seen previously in calculus though, here we do things the other way around: we start from a sequence and try to find its generating function.

Definition 5.14

The **generating function** (GF) associated to the sequence $(a_n)_{n\in\mathbb{N}}$ is a function G which can be written as a formal power series with the terms of the sequence as coefficients:

$$G(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n + \ldots = \sum_{n=0}^{+\infty} a_n x^n$$

The series itself is called a **generating series**.

Remark

We used the term *formal* because the series is only used as a representation of the sequence. x shouldn't be seen as an actual variable taking real values, we only use x^n as a symbol that marks, or corresponds to "where" the n^{th} term of the sequence is in the series. It is only when we state the equality of a generating series with a function that the associated "real" series should be well-defined, that is, it should converge for at least some values of x.

Because we have this abstraction of a formal power series, we can redefine how to add or multiply them together.

Definition 5.15

Let $f(x) = \sum_{n=0}^{+\infty} a_n x^n$ and $g(x) = \sum_{n=0}^{+\infty} b_n x^n$. Then

$$\text{(i)}\ f(x)+g(x)=\sum_{n=0}^{+\infty}(a_n+b_n)x^n, \quad \text{(addition)}$$

$$\text{(ii)}\ f(x)g(x)=\sum_{n=0}^{+\infty}\left(\sum_{j=0}^{k}a_j b_{k-j}\right)x^n, \text{(multiplication)}$$

$$\text{(iii)}\ x^m f(x)=\sum_{n=m}^{+\infty}a_{n-m}x^n \qquad \text{(shifting)}$$

Remark

This actually matches the algebra of actual power series, simply because we defined it so it matches, since we want to eventually interpret the series as a closed-form function.

A summary of known series which can be identified when determining a closed form from a generating series is shown in Table 5.1.

Power series	Closed form	Reference
$\sum_{n=0}^{k}\binom{k}{n}x^n$	$(1+x)^k$	Theorem 4.26
$\sum_{n=0}^{k}x^n$	$\frac{1-x^{k+1}}{1-x}$	Proposition 5.8
$\sum_{n=0}^{+\infty}a^n x^n$	$\frac{1}{1-ax}$	Taylor
$\sum_{n=0}^{+\infty}\frac{x^n}{n!}$	e^x	Taylor
$\sum_{n=1}^{+\infty}\frac{(-1)^{n+1}}{n}x^n$	$\log(1+x)$	Taylor

Table 5.1: Some useful closed forms for power series

5.4.2 Solving recurrence relations

An example of the power of generating functions is how they can be used to solve recurrence relations, using the following procedure:

(i) Rewrite the recurrence relation for a_n in terms of an equation that only involves the generating function G.
(ii) Solve this equation and obtain a closed form for $G(x)$.
(iii) Compute the coefficients a_n by performing the Taylor expansion of G around $x=0$.

Example. Solve the recurrence relation $\forall n\in\mathbb{N}^*, a_n=3a_{n-1}$, with $a_0=2$.

5.4.3 Solving counting problems

A generating series can encode a specific kind of sequence: namely, the sequence (a_n) of number of ways to select some object n times from a set. So for instance, the series $(1 + 2x + 3x^2)$ can encode the fact that we have a single way to not select the object, two to select it once, and three to select it twice.

Example. Let's say we want to determine the number of ways to insert tokens worth 1€, 2€ and 5€ into a vending machine to pay for an item that costs r euros, when the order in which the tokens are inserted does not matter.

Solution. The number of euros produced by picking n tokens worth i euros can be seen as the exponent in $(x^i)^n = x^{in}$ within the formalism of generating functions. Besides, we might pick any number of tokens of each type, so for each type we need to sum over all possible number of picks n. Then the answer is the coefficient in front of x^r in the following generating function:

$$\left(\sum_{n=0}^{+\infty} x^n\right)\left(\sum_{n=0}^{+\infty} x^{2n}\right)\left(\sum_{n=0}^{+\infty} x^{5n}\right)$$

using the product rule. The independence of picks is clearly visible in the generating function.

When the order matters, then using the product rule, the process of picking n tokens successively can be represented by $(x + x^2 + x^5)^n$. The number of ways for these n picks to sum up to r euros is then the coefficient of x^r. Since any number of tokens may be inserted, the number of ways to produce r euros when the order matters is the coefficient of x^r when we sum over all possible n values:

$$\sum_{n=0}^{+\infty} (x + x^2 + x^5)^n$$

In both cases, the answer might seem complicated to get by hand for large r, but it is relatively straightforward using a computer.

> **Remark**
>
> In generating functions representing combinatorial problems, you can think of additions as logical "or", and products as logical "and".

> **Remark**
>
> Here, we barely scratched the surface of how generating functions can be used for counting problems. They can go as far as pairing together with complex number analysis to solve some hard counting problems.

6 Graph theory

This chapter overlaps with sections 10.1-4, 10.6, 11.1 and 11.4-5 of Rosen.

Why study graphs? In general, because they help you understand the structure on which some interactions play out. But for now, it's enough to say: *because they look nice*, whether they represent mobility flows, online social networks, trade networks or the internet.

6.1 Defining graphs

6.1.1 Basic definitions

Definition 6.1 (Directed graphs)

A **directed graph** is an ordered pair $G = (V, E)$ where
- V is a nonempty set of **vertices**, also called nodes.
- $E \subseteq V \times V$ is the graph's adjacency relation, which is a binary relation on V. Its elements are called **edges**, or links. If $e = (u, v) \in E$, we say that e is **incident** on u and v, that u is e's **tail** and v its **head**.

Definition 6.2

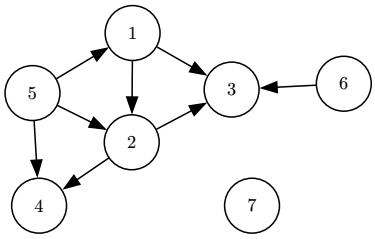
In a directed graph $G = (V, E)$, for all $u \in V$ we define:
- its set of predecessors or its **in-neighbourhood** as:

$$\mathcal{N}^{(\text{in})}(u) = \{v \in V \mid (v, u) \in E\},$$

- its set of successors or its **out-neighbourhood** as:

$$\mathcal{N}^{(\text{out})}(u) = \{v \in V \mid (u, v) \in E\}.$$

We actually already introduced directed graphs as a natural representation of a binary relation, in Section 1.4! Directed graphs can then be represented in a similar way. For instance:



Definition 6.3 (Undirected graphs)

An **undirected graph** is an ordered pair $G = (V, E)$ of vertex and edge sets V and E, where E is a set of *unordered* pairs of vertices, that is $E \subseteq \{\{u, v\} \mid u, v \in V\}$. If $e = \{u, v\} \in E$, we say that e is **incident** on u and v.

Remark

The edge set of a directed graph is defined as

$$E \subseteq V \times V = \{(u, v) \mid u, v \in V\},$$

which is a set of *ordered* pairs of vertices.

Remark

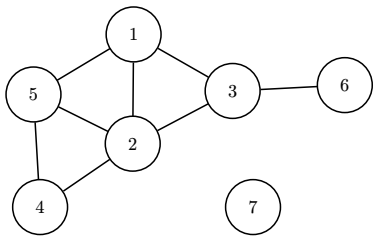
An undirected graph can be seen as a directed graph whose adjacency relation is symmetric – meaning if (u, v) is an edge, then (v, u) as well–, but also crucially considering that these are not two distinct edges but a single one, which can be represented as a 2-set $\{u, v\}$.

Definition 6.4

In an undirected graph $G = (V, E)$, for all $u \in V$ we define its set of adjacent vertices, or its **neighbourhood** as:

$$\mathcal{N}(u) = \{v \in V \mid \{u, v\} \in E\}.$$

The representation of undirected graphs then simplifies the edges by removing the arrows indicating direction:



In the following, we'll use the word **"graph"** to refer to either a directed or undirected graph.

Again in a similar fashion to relations, graphs can be represented by an adjacency matrix.

Definition 6.5 (Adjacency matrix)

Let's consider a graph $G = (V, E)$, and the ordering $v_1, v_2, ..., v_{|V|}$ of its vertex set V. The **adjacency matrix** of G associated to that ordering is the $|V| \times |V|$ matrix whose entries A_{ij} count the number of edges between v_i and v_j.

Property 6.6

The adjacency matrix of an undirected graph is symmetric.

Notation

In the following, whenever we refer to a vertex as the vertex i or j, we mean v_i or v_j in the ordering $v_1, v_2, ..., v_{|V|}$, from which the graph's adjacency matrix has been defined.

6.1.2 Particular graphs

Definition 6.7 (Complete graph)

A graph $G = (V, E)$ is said to be complete if E contains all edges which can be formed between vertices in V.

Question

How many edges do complete (un)directed graphs have?

An interesting particular case of graphs is the one that represents relationships between two separate sets of entities.

Definition 6.8 (Bipartite graphs)

A graph $G = (V, E)$ is **bipartite** if its vertex set V can be partitioned into two disjoint subsets V_1 and V_2 such that every edge in the graph connects a vertex in V_1 with a vertex in V_2.

Remark

A directed bipartite graph can actually represent a binary relation between two sets.

We will now introduce some particular kinds of graphs which are not so common and that we will largely ignore in the following.

Definition 6.9 (Self-loops)

In a graph $G = (V, E)$, a vertex which is adjacent to itself is called a self-loop.

Remark

A graph can either allow or disallow self-loops, depending on what it models. Following the binary relation analogy, disallowing self-loops can be thought of as forcing the adjacency relation to be irreflexive.

The entries of the adjacency matrix defined in Definition 6.5 have been defined as edge *counts*, meaning it could take values in $\mathbb{N}$, while graph definitions above implied values in $\{0, 1\}$, encoding the absence or presence of an edge. Actually, we defined adjacency matrices this way so it's general enough to account for multigraphs.

Definition 6.10 (Multigraphs)

A **multigraph** $G = (V, E)$ is a graph in which multiple edges are allowed between the same pair of vertices, which implies that E is not defined as a set but as a collection with potential repetitions.

Multigraphs are actually very rare in practice, as they can often be equivalently represented by weighted graphs.

Definition 6.11 (Weighted graphs)

A **weighted graph** $G = (V, E, \omega)$ is a graph such that every edge $e \in E$ is associated to a weight $\omega(e) \in \mathbb{R}$.

Remark

Naturally then, the entries of the adjacency matrix of a weighted graph correspond to the edge weights, that is:

$$A_{ij} = \omega\big((v_i, v_j)\big) = \omega(ij)$$

Most of the time in this course, we will ignore these generalisations and only consider simple graphs.

Definition 6.12 (Simple graphs)

A simple graph is an unweighted graph without multiple edges or self-loops.

Since graphs are defined from a pair of sets, the notion of subset also naturally exists in graphs, with subgraphs.

Definition 6.13 (Subgraphs)

The graph $H = (W, F)$ is a **subgraph** of $G = (V, E)$ if its vertex and edge sets are subsets of G's, that is $W \subseteq V$ and $F \subseteq E$.

6.1.3 Isomorphism of graphs

The same graph can be represented in a number of ways: graphically the vertices are placed arbitrarily on a plane, and in an adjacency matrix, the ordering of vertices is arbitrary. Also, if the vertex labels do not carry any particular meaning –i.e. they are "dummy" labels–, the labels too are arbitrary. It is thus important to be able to recognize the same graph, however it was represented.

Definition 6.14 (Graph isomorphism)

The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are **isomorphic** if and only if there exist a bijective function $f : V_1 \to V_2$ with the following property: a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2. The function f is called an **isomorphism**.

Some simple tests based on aggregate measures can be used to discard isomorphism between two graphs.

Remark

Given two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, then
(i) If $|V_1| \neq |V_2|$, then G_1 and G_2 are not isomorphic.

(ii) If $|E_1| \neq |E_2|$, then G_1 and G_2 are not isomorphic.

Remark

G_1 and G_2 are isomorphic if there exists an invertible linear map (basically a permutation of the "basis vectors") $\pi : V_1 \to V_2$ such that $A_2 = P^{-1} \cdot A_1 \cdot P$.

There are $|V_1|! = |V_2|!$ maps of this type, hence the difficulty of testing for isomorphism if two graphs share many similarities at the aggregate level!

6.2 Traversing graphs

The whole point of a graph model is not to only look at binary relationships between pairs of entities represented by edges between pairs of vertices, but to investigate the interconnections over the whole set of vertices, thus including "long-distance relationships". That is why traversing graphs is crucial, as it can provide us with central information about a graph.

6.2.1 Definitions

Definition 6.15 (Walks on graphs)

A **walk** on a graph $G = (V, E)$ is an alternating sequence of vertices and edges of the form $v_0, e_1, v_1, e_2, v_2, \ldots, v_{l-1}, e_l, v_l$, such that e_k is from v_{k-1} to v_k for all k. It is said to be **closed** if it ends where it starts, so if $v_0 = v_l$, and open otherwise. The **length** of the walk is equal to the number of edges in the walk l, and is at least one.

Remark

- In an undirected graph, the condition on each edge is $e_k = \{v_{k-1}, v_k\}$, meaning that edges can be traversed in both directions.
- A walk on a graph without multi-edges can be described more simply by a sequence of vertices, as each edge is uniquely determined by a pair of vertices.

Question

What is the smallest closed walk possible? Under what condition on G can it exist?

Definition 6.16

A **trail** is a walk with no repeated edge. A closed trail is called a **circuit**.

A **path** is a trail with no repeated vertex, with the potential exception of the first which may also appear as the last in the sequence. A **cycle** is a closed path.

Remark

If a walk has a repeated edge, then it has a repeated vertex, but the contrary is not necessarily true.

Paths allow us to determine a binary relation between vertices: whether they're connected, or not.

Definition 6.17 (Vertex pair connectedness)

Given a graph $G = (V, E)$ and two vertices $u, v \in V$, we say that u is **connected** to v if $u = v$, or if there exists a path which starts at u and ends at v.

Question

In which cases is this relation an equivalence? A partial order?

$$\forall u, v \in V, u \, R \, v \Leftrightarrow u \text{ is connected to } v$$

6.2.2 Shortest paths

A vertex can be connected to another, but this does not tell us how easy it is to go from one to the other, or, in other words, how distant the destination is from the origin. This distance could simply be linked to the lengths of the paths connecting them. But here we'll be a bit more general, and assume that some edges might be more costly to traverse than others. So let's generalise the notion of path length to weighted graphs.

Definition 6.18 (Path length)

Given a simple weighted graph $G = (V, E, \omega)$ with positive weights, the length $L(P)$ of a path $P = v_0, e_1, v_1, e_2, v_2 \ldots, v_{l-1}, e_l, v_l$ is the sum of the weights of the edges that it traverses:

$$L(P) = \sum_{i=1}^{l} \omega(e_i).$$

Here, the edge weights are thus interpreted as a distance between the vertices they connect. This definition allows to recover the one of walk length for unweighted graphs (Definition 6.15) by taking unit weights.

Definition 6.19 (Shortest path problem)

Given a simple weighted graph $G = (V, E, \omega)$ with positive weights, and $s, t \in V$, a shortest path problem consists in finding the shortest path that connects s and t, that is:

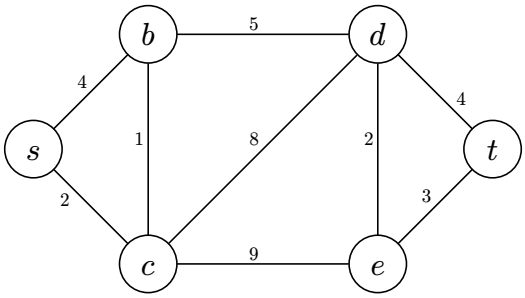
$$\operatorname*{argmin}_{P \in \mathcal{P}(s,t)} L(P),$$

where $\mathcal{P}(s, t)$ is the set of paths that connect s and t.

The shortest path length is then called the **distance** between s and t and is denoted $d(s, t)$.

Solving this problem is not obvious and requires us to actually traverse the graph using an algorithm, such as Dijkstra's. The basic idea of the algorithm is to iteratively explore the graph from the point of view of the starting vertex s. Starting from s, in each step we find its next closest vertex, until we've reached t. Crucially though, since shortcuts can sometimes appear, we need to keep track of known or estimated distances throughout the exploration. Let's proceed on the following example.

Example. Find the shortest path from s to t in:



Algorithm 6.1: Dijkstra's

```
1  procedure Dijkstra(G = (V, E, ω) with V = {s = v_1, ..., v_n = t})
2      for all v ∈ N^(out)(s)
3          d(s, v) = ω(s, v)                          ▷ initialise known distances: to neighbours
4      for all v ∈ V \ N^(out)(s)
5          d(s, v) = ∞                                ▷ initialise unknown distances as infinity
6      D = ∅                                          ▷ set of vertices with known distance
7      while t ∉ D                                    ▷ until we've reached t
8          u = argmin_{v∈V\D} d(s, v)                 ▷ consider the next closest vertex
9          D = D ∪ {u}
10         for all v ∈ N^(out)(u) \ D                 ▷ for all neighbours v of u
11             if d(s, u) + ω(u, v) < d(s, v) then    ▷ if going through u creates a shortcut to reach v
12                 d(s, v) = d(s, u) + ω(u, v)        ▷ update the estimated distance to v
13     return d(s, t)                                 ▷ shortest path length from source s to target t
```

Remark

- If at Line 8 there are several options, we can choose any of them.
- The sequence of vertices composing the path can be obtained by keeping track of the current best path, or simply of the best predecessor, whenever we reassign $d(a, v)$ in Line 12.
- The shortest path between two vertices is not necessarily unique. It is perfectly possible to have two or more paths of equal length between a given pair of vertices.

6.3 Characterising graph elements

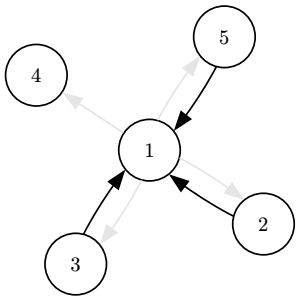
6.3.1 Degree centrality

A vertex' importance can first be quantified by its degree.

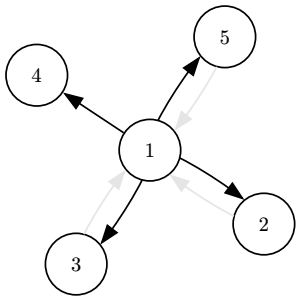
Definition 6.20 (Vertex in/out-degree)

Let G be a directed graph $G = (V, E)$, and let $u \in V$.

- The **in-degree** $k^{(\text{in})}(u)$ of u is the number of edges whose head is u, that is, which end at u. In other words, $k^{(\text{in})}(u) = \left|\mathcal{N}^{(\text{in})}(u)\right|$.
- The **out-degree** $k^{(\text{out})}(u)$ of u is the number of edges whose tail is u, that is, which start from u. In other words, $k^{(\text{out})}(u) = \left|\mathcal{N}^{(\text{out})}(u)\right|$.



Computing 1's in-degree



Computing 1's out-degree

Proposition 6.21

Let's consider a directed graph G and its adjacency matrix A associated to the ordering $v_1, v_2, ..., v_{|V|}$ of its vertex set V. Then:

$$\forall i \in [\![1, |V|]\!], \begin{cases} k^{(\text{in})}(v_i) = \sum_j A_{ji} \\ k^{(\text{out})}(v_i) = \sum_j A_{ij} \end{cases}$$

It then follows that summing over degrees amounts to summing all entries of the adjacency matrix, which gives the following result.

Proposition 6.22

In any directed graph $G = (V, E)$:

$$\sum_{v \in V} k^{(\text{in})}(v) = \sum_{v \in V} k^{(\text{out})}(v) = |E|$$

The definition of degree follows naturally for undirected graphs, by considering that two directed edges (u, v) and (v, u) correspond to a single undirected edge $\{u, v\}$.

Definition 6.23 (Vertex degree)

The **degree** of a vertex $u \in V$ in an undirected graph $G = (V, E)$ is the number of edges incident with it, except that a loop contributes twice to the degree of that vertex. The degree of a vertex u is denoted by $k(u)$.

Proposition 6.24

Let's consider an undirected graph G and its adjacency matrix A associated to the ordering $v_1, v_2, ..., v_{|V|}$ of its vertex set V. Then:

$$\forall i \in [\![1, |V|]\!], k(v_i) = A_{ii} + \sum_j \frac{A_{ij} + A_{ji}}{2} = A_{ii} + \sum_j A_{ij}$$

And, as a special case of Theorem 6.37:

Proposition 6.25

Let's consider an undirected graph G and its adjacency matrix A associated to the ordering $v_1, v_2, ..., v_{|V|}$ of its vertex set V. Then

$$\forall i \in [\![1, |V|]\!], \quad k(v_i) = (A^2)_{ii}$$

We can then get a similar result as Proposition 6.22, which is known as the handshaking theorem.

Theorem 6.26 (The handshaking theorem)

In any undirected graph $G = (V, E)$, we have that

$$\sum_{v \in V} k(v) = 2|E|$$

Remark

This also holds for undirected graphs with loops, thanks to the convention we took to consider a self-loop as incident twice to its vertex.

Corollary 6.26.1

For any undirected graph, the sum of all vertex degrees is an even number.

Corollary 6.26.2

Any undirected graph has an even number of vertices of odd degree.

Corollary 6.26.3

For any undirected graph with an odd number of vertices, there is an odd number of vertices of even degree.

The notion of degree also allows us to define a very special kind of graph.

Definition 6.27 (Regular graph)

A **k-regular graph** is an undirected graph whose vertices all have the same degree k.

The notion of degree also gets naturally generalised to the case of weighted graphs.

Definition 6.28 (Vertex in/out-strength)

Let G be a weighted directed graph $G = (V, E, \omega)$, and let $u \in V$. The **in-strength** $s^{(\text{in})}(u)$ of u is the sum of the weights of edges whose head is u. The **out-strength** $s^{(\text{out})}(u)$ of u is the sum of the weights of edges whose tail is u.

Definition 6.29 (Vertex strength)

Let G be a weighted undirected graph $G = (V, E, \omega)$, and let $u \in V$. The **strength** $s(u)$ of u is the sum of the weights of edges adjacent to u, except that a loop contributes its weight twice.

Proposition 6.21 then also holds for strengths in weighted directed graphs, and Proposition 6.24 for those in weighted undirected ones.

These measures, degree and strength, are very local in nature, and thus may not represent the importance of a vertex in the context of the whole graph.

6.3.2 Eigenvector centrality

A first way to provide more global information to quantify a vertex' importance is to consider that a vertex should be considered more important not only the more neighbours it has, but also the more important these neighbours are.

That is how we define the eigenvector centrality $x(u)$ of a vertex u in an undirected graph: it's proportional to the sum of the centralities of its neighbours:

$$x(u) = c^{-1} \sum_{v \in \mathcal{N}(u)} x(v),$$

where $c \in \mathbb{R}^+$ is a constant.

Rewriting this equality for every vertex, and involving the adjacency matrix A of the graph, we get

$$x = c^{-1} Ax \Leftrightarrow Ax = cx,$$

where x is a vector of centrality scores, which is an eigenvector of A associated to the eigenvalue c! And from the Perron-Frobenius theorem from linear algebra, we know that since A has non-negative values, if we want our centralities to all be non-negative, c must be the largest eigenvalue.

Definition 6.30 (Eigenvector centrality)

Given a simple undirected graph represented by an adjacency matrix A, the eigenvector centrality of vertex i is the i^{th} element of one of A's leading eigenvectors.

Remark

- The point of centralities is to compare vertices, so the same eigenvector should be considered for all of them!
- To generalise to directed networks, some small tweaks are needed: see Sections 7.1.2-3 in Newman.

6.3.3 Betweenness centrality

A second way to account for the global graph context to quantify a vertex' centrality is to use shortest paths.

Definition 6.31 (Vertex betweenness: simple)

The betweenness centrality of a vertex is the number of shortest paths between pairs of other vertices which pass through it.

However, as mentioned above, there may be more than one shortest path between a pair of vertices.

Definition 6.32 (Vertex betweenness: full)

The betweenness centrality of a vertex is the number of shortest paths between pairs of other vertices which pass through it, where we weight each path by the number of shortest paths between each pair of vertices:

$$\forall u \in V, b(u) = \sum_{s,t \in V \setminus \{u\}} \sum_{P \in \mathcal{P}(s,t)} \sum_{v \in P} \frac{\delta_{uv}}{|\mathcal{P}(s,t)|},$$

where $\mathcal{P}(s,t)$ is the set of shortest paths from s to t.

A strength of betweenness is that it not only characterises the importance of vertices, but of edges too!

Definition 6.33 (Edge betweenness)

The betweenness centrality of an edge is the number of shortest paths passing through it, where we consider paths between all connected vertex pairs, weighting multiple shortest paths appropriately.

Remark

This edge centrality is linked to the "strength of weak ties" theory formulated by the sociologist Mark Granovetter, which suggests that weak social relationships are central for the diffusion of information in a network (in very rough terms).

6.4 Describing graph structure(s)

6.4.1 Connectedness

The binary relation between vertices we've seen in Definition 6.17 can be used to perform a similar binary characterisation, but about the whole graph.

Definition 6.34 (Graph connectedness)

An undirected graph is said to be **connected** if every one of its vertices is connected to every other. A disconnected graph can be partitioned into connected subgraphs, also known as **connected components**.

Definition 6.35

A directed graph $G = (V, E)$ is

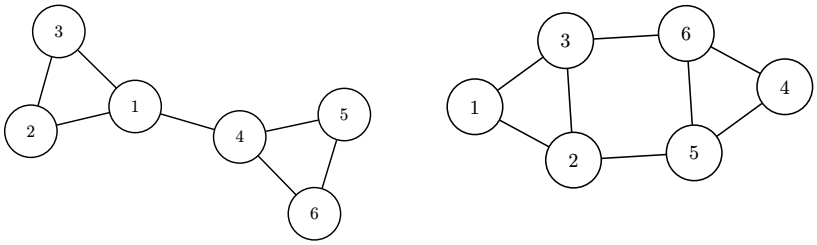
- **strongly connected** if every one of its vertices is connected to every other.
- **weakly connected** if its underlying undirected graph is connected.

A strongly or weakly disconnected graph can then be partitioned into **strongly and weakly connected components**.

6.4.2 Connectivity

Question

The connectedness only tells us whether a graph is connected, but not how strongly it is. But given the following two graphs, which would you say is more strongly connected? Why?



One way to quantify a graph's connectivity is to check how easy it is to split a graph into separate components.

Definition 6.36

A **cut edge** or **bridge** of a graph G is an edge whose removal produces a graph with more connected components than in G.

A **cut vertex** or **articulation point** of a graph G is a vertex whose removal (together with those edges incident with it) produces a graph with more connected components than in G.

Counting the minimum number of cut edges/vertices necessary to create k components thus allows a better characterisation of a graph's connectivity, as it is not only binary (is or is not connected) but quantified (a number saying how well-connected it is).

When looking at individual vertices instead, their connectivity can be quantified by counting the number of walks which connect them.

Theorem 6.37

Let G be a graph with adjacency matrix A with respect to the ordering $v_1, v_2, \ldots, v_{|V|}$ of its vertex set. The number of distinct oriented walks of length $l \geq 1$ that start at v_i and end at v_j is given by the entry (i, j) of the matrix A^l.

Proof: This can be proven by induction on the walk length l.

Basis step the number of walks from v_i to v_j of length 1 is simply 1 if $(v_i, v_j) \in E$, and 0 otherwise, so it is simply a_{ij}, for every i and j.

Inductive step let's consider a fixed l, write $B = A^l$, and assume that b_{ij} gives the number of distinct walks of length l from v_i to v_j.

A walk of length $l+1$ from v_i to v_j is made up of a walk of length l from v_i to some intermediate vertex v_k, and an edge from v_k to v_j. By the product rule, the number of such walks is the product of the number of walks of length l from v_i to v_k, which is b_{ik}, and the number of edges from v_k to v_j which is a_{kj}.

By the sum rule over possible values of k, the number of such walks of length $l+1$ is thus

$$\sum_{k=1}^{|V|} b_{ik}a_{kj}$$

Also, writing $C = A^{l+1} = A^l A$:

$$c_{ij} = \sum_{k=1}^{|V|} b_{ik}a_{kj}$$

The result is thus true for $l+1$ too, so by induction, it is for any length $l \geq 1$.

□

Question

How would this help us getting shortest path lengths in an unweighted graph?

Remark

Connectivity between two vertices is better characterised by computing the number of *paths* joining them. However, counting paths is a **much** harder problem, as illustrated in this life-changing video.

6.4.3 Graph size

The size of a graph can be simply considered to be its number of vertices and edges, but this gives little information. Indeed, it does not say how far away vertices are from each other in the graph, or, in other words, how hard it is to traverse the graph. For that, we can look into the lengths of shortest paths, and for instance compute the average shortest path length.

Alternatively, we can consider its longest shortest path.

Definition 6.38 (Graph diameter)

The diameter of a graph is the length of its longest shortest path, or in other words, the distance between its farthest vertices, in terms of shortest path length.

Remark

The graph diameter is what's behind the idea of the "six degrees of separation": the fact that if you build the graph of social connections between people, its diameter is equal to 6 (or at least, in a famous experiment made by Stanley Milgram, for more see this video).

6.4.4 Graph density

We already saw how to check how well connected a graph is, using cut edges and vertices (Definition 6.36). A complementary, easier way to quantify that is to count how many edges the graph has, and compare it to the maximum it could have.

Definition 6.39 (Graph density)

The density ρ of a graph is the fraction of possible edges which are actually present in the graph. In other words, it is equal to the number of edges present in the graph divided by the number of edges of the corresponding complete graph.

Remark

The density can be thought of as the probability that two vertices picked from the graph uniformly at random are connected by an edge.

Proposition 6.40

For a simple undirected graph $G = (V, E)$,

$$\rho = \frac{2|E|}{|V|(|V|-1)}.$$

6.4.5 Clustering

What if we now wish to quantify the density of connections around a specific vertex? The simplest way, similar to the idea of density, would consist in computing the fraction of possible edges the vertex formed. This characterisation would be exactly equivalent to simply considering the vertex degree, though. More interesting would be to also quantify how well-connected the neighbours of a vertex neighbours are. In terms of social networks, this is saying how much the "the friend of my friend is my friend" rule holds. Let's start with a couple of definitions to help us think about this.

Definition 6.41 (Triads and triangles)

Given a simple undirected graph $G = (V, E)$

- a **triad** is a connected subgraph of G which contains three vertices,
- a **triangle** is a triad which is complete.

Proposition 6.42

- Considering a vertex $u \in V$ of degree at least 2, the subgraph comprising u and two of its neighbours is by definition a triad. It is a triangle only if these two neighbours are connected themselves.
- Three vertices of G form a triad if and only if they are connected by a path of length 2. Triangles are thus cycles of length 3, which we can also call *closed triads*.

We thus wish to quantify the tendency for *triadic closure* around each vertex.

Question

Which concept from binary relations is this related to?

For a single vertex, this is quantified by the local clustering coefficient.

Definition 6.43 (Local clustering coefficient)

Given a simple undirected graph $G = (V, E)$, the local clustering coefficient C_u of a vertex $u \in V$ is the proportion of triads it is a part of which are closed.

Proposition 6.44

Given an adjacency matrix A for G and a vertex v_i such that $k(v_i) \geq 2$,

$$C_{v_i} = \frac{\sum_j \sum_{l<j} A_{ij} A_{jl} A_{li}}{\binom{k(v_i)}{2}}.$$

Remark

A vertex with low clustering can be seen as central in its neighbourhood, since it means information flowing in this neighbourhood would need to pass through it to propagate. Local clustering can thus be seen as a more local version of betweenness, as the latter measures the importance of a vertex for the flow of information over the whole connected graph.

To quantify this tendency across the whole graph, one could first average local clustering coefficients. This is not always the best choice though, as it gives the same importance to all vertices, irrespective of their degree. An alternative is to compute the following:

Definition 6.45 (Global clustering coefficient)

Given a simple undirected graph $G = (V, E)$, the global clustering coefficient C of G is the fraction of triads which are closed in the whole graph.

Proposition 6.46

Given an adjacency matrix A for G,

$$C = \frac{\sum_i \sum_j \sum_l A_{ij} A_{jl} A_{li}}{\sum_i \sum_j \sum_{l \neq i} A_{ij} A_{jl}}.$$

6.4.6 Cliques and cores

We might also go one step further, and wonder if the friend of two of my friends also tends to be my friend. Or, any number of steps further, using the following generalisation.

Definition 6.47 (k-cliques)

Given a simple undirected graph $G = (V, E)$, a k-clique ($k \geq 2$) is a subgraph of G which contains k vertices and is complete.

Cliques in a graph thus define highly cohesive groups of vertices, which can be of any size.

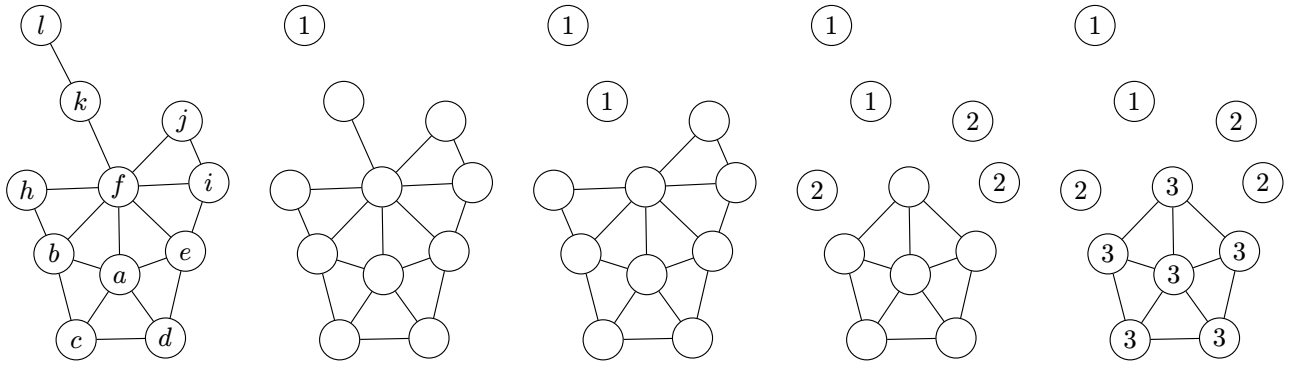
Another useful generalisation resides not in the size of the subgraph, but in how densely connected it is.

Definition 6.48 (k-cores)

Given a simple undirected graph $G = (V, E)$, a k-core is a subgraph of G whose vertices are all adjacent to at least k others.

This decomposition is sometimes used to detect core–periphery structures in networks: nodes that lie within the highest-k-cores are "core" nodes within the network, while nodes outside those cores are "peripheral" nodes.

A graph can be decomposed into k-cores relatively easily, by iteratively removing vertices of increasing degrees, as shown below.



Algorithm 6.2: k-core decomposition

procedure k-core($G = (V, E)$: connected undirected graph)
  k = 1
  while G is not empty
    $V_k = \emptyset$ ▷ set of vertices in k-core
    while $\min_{u \in V}(k(u)) \leq k$
      $V_k = V_k \cup \{u \in V \mid k(u) \leq k\}$ ▷ identify vertices of degree k or less
      $V = V \setminus V_k$ ▷ remove these vertices and their edges from the graph
      $E = E \setminus \{\{u, v\} \in E \mid (u \in V_k) \vee (v \in V_k)\}$
    $k = k + 1$
  return $\{V_1, V_2, \ldots V_{k_{\max}}\}$ ▷ k-core decomposition

6.5 Trees

A tree is a special kind of graph which is useful in many applications, especially in computer science.

6.5.1 Definitions

Definition 6.49 (Tree)

A **tree** is a simple connected and undirected graph with no cycles. A **forest** is a simple undirected graph with no cycles. Each connected component of a forest is a tree.

The natural way to represent a tree is to pick one of its vertices as its root, and place it at its top, with a branching structure going down.

Definition 6.50 (Rooted tree)

A **rooted tree** is a tree T in which one vertex has been designated as the **root** and every edge is directed away from this root.

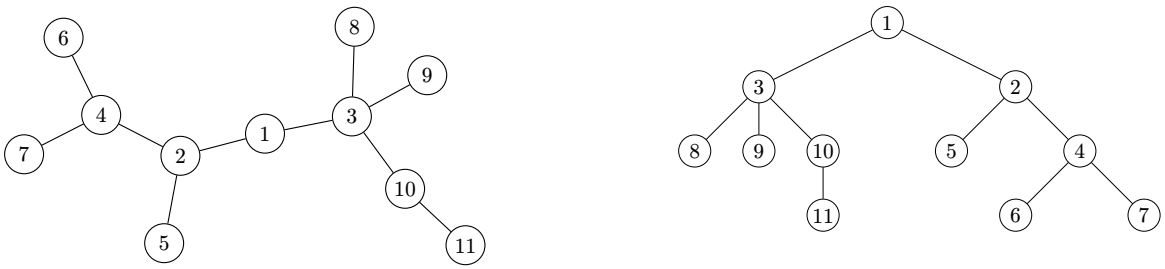
Definition 6.51 (Rooted tree vertices)

Let's consider a rooted tree T. If v is a vertex in T other than the root, the **parent** of v is the unique vertex u such that there is a directed edge from u to v. The vertex v is then a **child** of u.

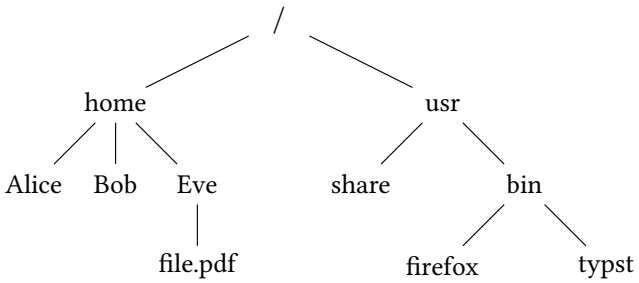
A vertex without children is called a **leaf**, and all others are called **internal vertices**.

When drawing a rooted tree, its root is placed at the top, and children placed below their parents. Thus, the arrows indicating the edge directions all point down, and can be omitted.

Example. Here is how the same tree can be represented when non-rooted (left) and when rooted on 1 (right).



Example. Computer file systems are organized as trees with a natural root called the root directory `/`.



What do leaves and internal vertices correspond to in this case?

The most important property of trees follows.

Theorem 6.52

A graph $G = (V, E)$ is a tree if and only if there exists a unique path between any pair of vertices.

It comes naturally from the fact that if we add a second path between any pair of vertices of a tree, we create a cycle. It is central because it means many problems are very simple on trees, notably those related to shortest paths.

Theorem 6.53

(i) The simple graph G is a tree if and only if it is connected and, if we remove any edge, we obtain a disconnected graph.
(ii) The simple graph G is a tree if and only if it does not contain any cycles and, if we add any edge without adding any vertex, we create a cycle.

A tree can therefore only be grown by adding a vertex and an edge connecting an existing vertex to this new one. More specifically, to grow a tree:
(i) Start from the trivial tree $T = (\{r\}, \emptyset)$, where r is the root vertex.
(ii) Given $T = (V, E)$, add a new vertex u and a new edge $\{u, v\}$ where $v \in V$.

Theorem 6.54

Any graph obtained by using the preceding procedure is a tree, and any tree can be obtained in this way.

The validity of this growing procedure also leads to the following property, which follows by induction.

Theorem 6.55

Any tree with n vertices has $n - 1$ edges.

And the converse is also true! If we suppose a connected graph with n vertices and $n - 1$ edges is not a tree, then it means we can remove edges from this graph to destroy existing cycles, and thus create a tree, which would have fewer than $n - 1$ edges, which contradicts the above theorem.

Theorem 6.56

If G is a simple undirected graph with n vertices, then the following statements are equivalent:
(i) G is a tree.
(ii) G is connected and has $n - 1$ edges.
(iii) G has $n - 1$ edges and does not contain any cycle.

6.5.2 Spanning trees

Definition 6.57 (Spanning tree)

A **spanning tree** of a connected graph G is a subgraph of G that is a tree containing all vertices of G.

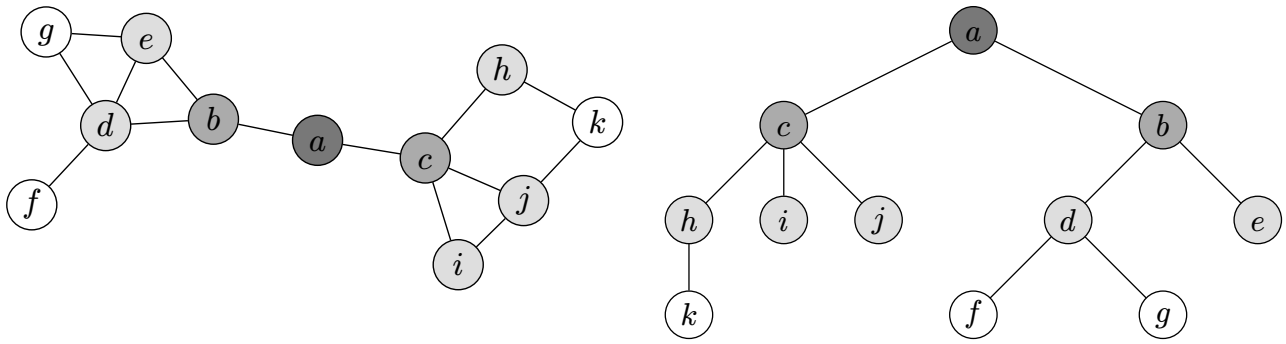
The most basic way to build a spanning tree is to remove edges from a graph to destroy its cycles, without removing vertices. This idea is the basis on which we can prove the following.

Theorem 6.58

A simple graph is connected if and only if it has a spanning tree.

Identifying cycles can be computationally hard, hence the need for other algorithms, such as breadth-first search. The idea of the algorithm is to explore the graph iteratively, starting from a root vertex. At each step, we consider all the neighbours of the vertices added to the tree in the previous step. A neighbour and its associated edge are then added to the spanning tree only if the vertex is not already present.

The reason it's called breadth-first can be directly understood from the figure below: at each step shown in a distinct shade of grey, we add all vertices of a given level to the tree, thus obtaining its full "breadth" at this level.



It can be formally written as follows.

Algorithm 6.3: Breadth-first search (BFS)

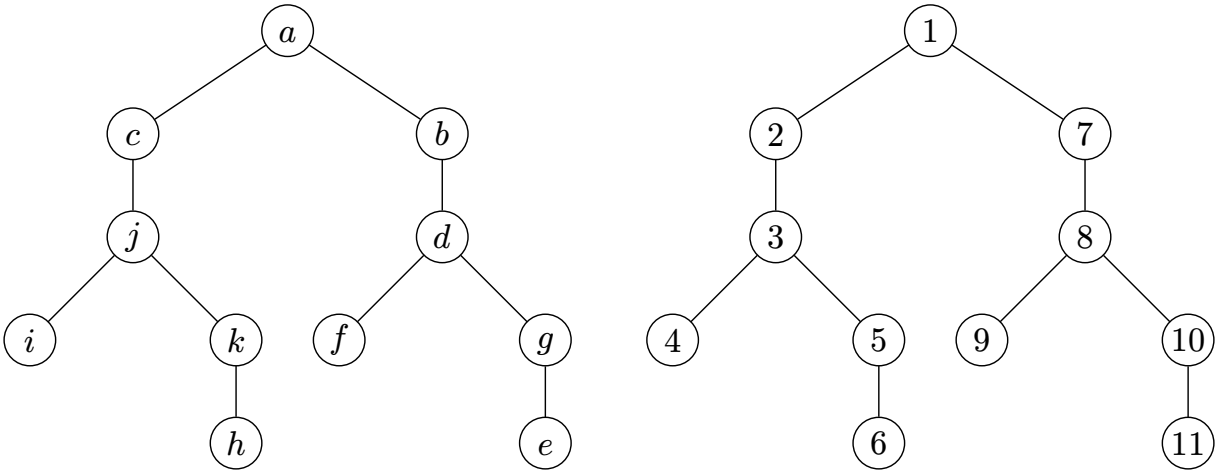
procedure BFS($G = (V, E)$: connected graph with $V = \{v_1, \ldots, v_n\}$)
  $T = (V_T, E_T) = (\{v_1\}, \emptyset)$ ▷ pick a root v_1 and initialise the tree
  $S_p = \{v_1\}$ ▷ set of vertices previously added to T
  while $|S_p| > 0$ ▷ while we have vertices to consider
    $S_n = \emptyset$ ▷ set of vertices to consider next
    for each $v \in S_p$ ▷ for each vertex previously added to T
      for each $w \in \mathcal{N}(v) \setminus V_T$ ▷ for all v's neighbours not already in T
        $T = (V_T \cup \{w\}, E_T \cup \{\{v, w\}\})$ ▷ add it to the tree
        $S_n = S_n \cup \{w\}$ ▷ consider w's neighbours at the next step
    $S_p = S_n$
  return T ▷ spanning tree

Remark

We already saw a variant of breadth-first search: Dijkstra's algorithm (Algorithm 6.1)!

We can also produce a spanning tree of a simple graph with depth-first search. The idea here is to form the tree by forming the longest path possible at each step. A first path starting from an arbitrary root is formed, until no more vertices can be added. The vertices and edges composing this path are added to the tree. We then backtrack through the path until we find a vertex from which we can start a second path. The same procedure as before is repeated, and the backtracking as well until all vertices are added to the tree.

We show on the figure below the tree resulting from a depth-first search carried out on the same graph as above. On the right-hand side vertex labels indicate the order in which they were added. The reason it's called depth-first can be directly understood from this figure: the tree is formed by forming branches which are as deep as possible, given the choices made during path formation.



Algorithm 6.4: Depth-first search (DFS)

```
1 procedure visit(v : v ∈ V)
2     for each w ∈ 𝒩(v) \ V_T                ▷ for each neighbour of v not already in T
3         T = (V_T ∪ {w}, E_T ∪ {{v, w}})    ▷ add it to the tree
4         visit(w)                           ▷ visit its neighbours
5 procedure DFS(G = (V, E): connected graph with V = {v_1, ..., v_n})
6     T = (V_T, E_T) = ({v_1}, ∅)            ▷ pick a root v_1 and initialise the tree
7     visit(v_1)
8     return T                               ▷ spanning tree
```

> **Remark**
>
> This algorithm is a very basic recursive algorithm, whose output (a growing tree), also turns out to be the way you can visualise *any* recursive algorithm. The reason is that every recursion depth level directly corresponds to a depth level of the rooted tree being grown.

Both algorithms can be used as the basis for algorithms that solve many different problems. For example, they can be used to find paths and circuits in a graph, to determine the connected components of a graph, or to find the cut vertices of a connected graph.

6.5.3 Minimum-weight spanning trees

A graph may admit many spanning trees, all with the same number of edges according to Theorem 6.55, so a priori equivalent. However, for weighted graphs, some might be more optimal, in a certain sense.

Definition 6.59

A **minimum-weight spanning tree** of a connected weighted graph $G = (V, E, \omega)$ is a spanning tree $T = (V, A)$ of G such that $\omega(A) = \sum_{e \in A} \omega(e)$ takes the minimum possible value.

Thus, if the weights encode some form of cost to traverse edges, as in the shortest path problem, finding a minimum-weight spanning tree gives an optimal way to fully traverse a graph.

Definition 6.60

A **greedy algorithm** to solve a given problem is an algorithm that when presented with a choice, always selects what seems to be the best option at this given moment.

Remark

There is no guarantee that selecting the best option at each step, so the one that's best *locally*, leads to a *globally* optimal solution in the end.

The two algorithms we will present below are greedy, but actually lead to an optimal solution!

The first is Prim's algorithm: it starts by adding any edge with smallest weight to the spanning tree. It then successively add edges of minimum weight that are incident to a vertex already in the tree, all the while avoiding to form cycles. It can stop once the tree is spanning, that is, once it has $|V| - 1$ edges, following Theorem 6.55.

Algorithm 6.5: Prim's algorithm

procedure Prim($G = (V, E, \omega)$: connected weighted graph with $V = \{v_1, v_2, ..., v_n\}$)
  $T = (V_T, E_T) = \left(\left\{v_{k_0}, v_{j_0}\right\}, \left\{\left\{v_{k_0}, v_{j_0}\right\}\right\}\right)$ where $\omega\left(\left\{v_{k_0}, v_{j_0}\right\}\right)$ is minimum.
  for $i = 1$ to $n - 2$
    $e_i = \left\{v_{k_i}, v_{j_i}\right\}$ edge of minimum weight that doesn't form cycles and such that $v_{k_i} \in V_T$.
    $T = \left(V_T \cup \left\{v_{k_i}\right\}, E_T \cup \{e_i\}\right)$
  return T ▷ minimum-weight spanning tree

But an image is worth a thousand words.

Theorem 6.61

Given a connected weighted graph $G = (V, E, \omega)$, Prim's algorithm produces a minimum-weight spanning tree of G.

Kruskal's algorithm, on the other hand, starts with a tree comprising all vertices. It then iterates through the list of edges sorted by weight, and adds it to the tree if it does not form a cycle. It stops when $|V| - 1$ edges have been added.

Algorithm 6.6: Kruskal's algorithm

```
1 procedure Kruskal(G = (V, E, ω): connected weighted graph
2     T = (V_T, E_T) = (V, ∅)
3     Place the edges by increasing weight into a list L
4     for each e in L
5         if e does not form a cycle when added to T
6             T = (V, E_T ∪ {e})
7     return T                                  ▷ minimum-weight spanning tree
```

Again, an image is worth a thousand words.

Theorem 6.62

Given a connected weighted graph $G = (V, E, \omega)$, Kruskal's algorithm produces a minimum-weight spanning tree of G.

Remark

In both cases,
- more than one edge may have the same weight,
- which means that the minimum-weight spanning tree might not be unique.

Check at home

There is much more to graph theory, and to its sister field of network science that we didn't cover in this course, such as: community detection, (random) graph models, random walks, more generally processes on graphs, etc.

You can check out the Complexity Explorables website for some visual introductions of these concepts. To go further, you can also use the following resources:
- Mark Newman's *Networks* book,
- Aaron Clauset's lectures notes,
- Michele Coscia's *Atlas for the aspiring network scientist*,

although many topics require knowledge in probability theory and statistics that we didn't require for this course.